```
            //1024 * 5 的乘法计算次数
        //fxet[i] =  (float) Math.abs((fxet[i] * dlt[i] + fxet[i + 1] * dlts[i])*(fxet[i] * dlts[i + 1] + fxet[i + 1] * dlt[i + 1]));
        //1024 * 4 的乘法计算次数
        //fxet[i] =  (float) Math.abs((fxet[i] * dlt[i] + fxet[i + 1] * dlts[i])+(fxet[i] * dlts[i + 1] + fxet[i + 1] * dlt[i + 1]));
        //1024 * 3 的乘法计算次数
            output[i] =  (float) Math.abs((input[i] * dlt[i] + input[i + 1] * dlts[i])*(dlts[i + 1] + dlt[i + 1]));
        //1024 * 2 的乘法计算次数
        //output[i] =  (float) Math.abs(input[i] * dlt[i] + input[i + 1] * dlts[i]);
        //1024 * 1 的乘法计算次数
        //output[i] =  (float) Math.abs(input[i] * dlt[i]);
          }
          return output;
        }

        public float[] doLdfft(float []input) {
             float[] fxet = input.clone();
             int size = input.length;
             int log = 2;
             float pi = (float) 3.1415926;
            //log - section - loop
            while(log < size) {
                double Qk = 2 * pi / log;//log2
                double currentC = Math.cos(Qk);
                double currentQic = currentC;
                double currentS = Math.sin(Qk);
                double currentQis = currentS;
                //loop
                double Ei = 0;
                for(int j = 0; j < log/2; j++ ) {
                    //常见教材中和差化积的表达如下
                    double regc = currentC * currentQic - currentS * currentQis;
                    currentS = currentC * currentQis + currentS * currentQic;
                    currentC = regc;
                    //section - loop
                    for(int i = 0; i + log < size; i = i + log) {
                    Ei = fxet[i + j] * currentC;
                    fxet[i + j] += Ei;
                    fxet[i + j + log/2] -= Ei;
                }
            }
            log = log * 2;
         }
        return fxet;
}
        //因为（2cosAcosB + 1) 除以2的频域误差性，
        //于是将时函数 的分数熵阶进行 极速处理 对应特殊对称2倍角（2cosAcosB - 1）的 逆代换优化
        public float[] do2Ldfft(float []input) {
            float[] fxet = input.clone();
    int size = input.length;
```

```
int log = 2;
float pi = (float) 3.1415926;
//这个for for循环init可以拿出去new， 提高实时计算效率
    double [][]cosNT = new double[input.length][input.length * 2];
            for(int i = 0; i < input.length; i++) {
                    for(int j = 2; j < input.length * 2; j*=2) {
                            cosNT[i][j] = Math.cos((i - 1) * pi / j);
                    }
            }
//log - section - loop
while(log < size) {
    double currentC;
   //loop
    double Ei = 0;
   for(int j = 0; j < log/2; j++ ) {
      //时函数 的分数熵阶进行 极速处理 对应特殊对称2倍角 （2cosAcosB - 1）的 逆代换 如下
      double detaT = cosNT[j * 2 + 2][log * 2];
      currentC = 2 * detaT * detaT - 1;
      //section - loop
     for(int i = 0; i + log < size; i = i + log) {
            Ei = fxet[i + j] * currentC;
            fxet[i + j] += Ei;
            fxet[i + j + log/2] -= Ei;
               }
   }
    log = log * 2;
}
for(int i = 0; i < fxet.length; i++) {
    fxet[i] = Math.abs(fxet[i]);
}
return fxet;
    }

    public float[] doEt2Ldfft(float []input) {
            float[] fxet = input.clone();
int size = input.length;
int log = 2;
float pi = (float) 3.1415926;
//这个for for循环init可以拿出去new， 提高实时计算效率
    double [][]cosNT = new double[input.length][input.length * 2];
            for(int i = 0; i < input.length; i++) {
                    for(int j = 2; j < input.length * 2; j*=2) {
                            cosNT[i][j] = Math.cos((i - 1) * pi / j);
                    }
            }
//log - section - loop
while(log < size) {
   //loop
    double Ei = 0;
```

```
        int logd = log >> 1;
        int logi = log << 1;
        //double detaT = cosNT[0][log * 2]; // 1/16
        for(int j = 0; j < logd; j++ ) {
        //去掉时函数的2倍角代换进行内核化T熵查询即可，
        double detaT = cosNT[j << 2][log << 1];
        //因为我设计内核是从0开始，而这里幂计算是1开始对2的倍数熵，
        //所以大家用于各类具体实际工程记得去校准一下，
           //section - loop
          for(int i = 0; i + log < size; i = i + log) {
                Ei = fxet[i + j] * detaT;
                fxet[i + j] += Ei;
                fxet[i + j + logd] -= Ei;
                //Ei = fxet[i + j] * detaT;
                //fxet[i + j] +=  fxet[i + j] * detaT;
                //fxet[i + j + logd] -= fxet[i + j+ logd] * detaT;
                   }
        }
        log = logi;
    }
    for(int i = 0; i < fxet.length; i++) {
        fxet[i] = Math.abs(fxet[i]);
    }
    return fxet;
        }
        //今天花了3个小时，剔除了2倍角的蝶形熵减，来满足计算加速，最后做1024的乘法，速度提高10倍，但质量降低10%，仅对2的指数倍频敏感，于是增加sin核做迪摩根补码。价值是多频调制解调，能精确控制某组频率的分贝值。
        public float[] doSigmaEt2Ldfft(float []input) {
        float[] fxet = input.clone();
        float[] fxetb = new float[input.length];
        int size = input.length;
        int log = 2;
        float pi = (float) 3.1415926;
        //这个for for循环init可以拿出去new， 提高实时计算效率
        float [][]cosNT = new float[input.length][input.length * 2];
        float [][]sinNT = new float[input.length][input.length * 2];
                for(int i = 0; i < input.length; i++) {
                        for(int j = 2; j < input.length * 2; j*=2) {
                                cosNT[i][j] = (float) Math.cos((i - 1) * pi / j);
                                sinNT[i][j] = (float) Math.sin((i - 1) * pi / j);
                        }
                }
                float detaT[][] = new float[size][size];
                float detaTs[][] = new float[size][size];
                float detaET[][] = new float[size][size];
                float detaETs[][] = new float[size][size];
    //log - section - loop
    while(log < size) {
```

```
        //loop
         double Ei = 0;
         int logd = log >> 1;
        int logi = log << 1;
        //double detaT = cosNT[0][log * 2]; // 1/16
        for(int j = 0; j < logd; j++ ) {
         //去掉时函数的2倍角代换进行内核化T熵查询即可，
         //double detaT = cosNT[j << 2][log << 1];
         detaT[j][log]= cosNT[j << 2][log << 1];
         detaTs[j][log]= sinNT[j << 2][log << 1];
         detaET[j][log]= 1 + detaT[j][log];
         detaETs[j][log]= 1 - detaT[j][log];
         //因为我设计内核是从0开始，而这里幂计算是1开始对2的倍数熵，
         //所以大家用于各类具体实际工程记得去校准一下，
            //section - loop
        }
         log = logi;
    }
    log = 2;
    float[] dlt = new float[input.length];
    float[] dlts = new float[input.length];
    for(int i = 0; i < fxet.length; i++) {
        dlt[i] = 1;
        dlts[i] = 1;
    }
    while(log < size) {
        //loop
         double Ei = 0;
         double Eis = 0;
         int logd = log >> 1;
        int logi = log << 1;
        for(int j = 0; j < logd; j++) {
            for(int i = 0; i + log < size; i = i + log) {
                    Ei = dlt[i + j] * detaT[j][log];
                    Eis = dlts[i + j] * detaTs[j][log];
                    dlt[i + j] += Ei;
                    dlt[i + j + logd] -= Eis;
                    //因为减熵是不规则的补码加乘逻辑，于是剔除，因为剔除了蝶形的倍频减熵，
                    //于是用sin来补充迪摩根互补律。
                    dlts[i + j] -= Eis;
                    dlts[i + j + logd] += Ei;
                       }
        }
         log = logi;
    }
    for(int i = 0; i < fxet.length; i++) {
         dlt[i] =  (float) Math.abs(2 * dlt[i] * dlt[i] - 1);
         dlts[i] =  (float) Math.abs(2 * dlts[i] * dlts[i] - 1);
    }
```

//上面的函数可以全部拿出去做定值全局，fft的o（nlogn） 次数在这里成了 o（n），切记只对2的指数峰敏感。老工程用记得修正 0 1 -1的计算偏移量和误区，新工程用记得重写测时函数。

```
    for(int i = 0; i < fxet.length - 1; i++) {
        fxet[i] =  (float) Math.abs(fxet[i] * dlt[i] + fxet[i] * dlts[i]);
    }
    return fxet;
}

//昨天思考了2个小时，关于log熵内积去时函数特殊关系和2倍角关系进行定值k提取，现在通过迪摩根的比特
//位计算进行归一化，我得到一个最简单的方式，
        public float[] doSigmaEt2Ldfft1(float []input) {
                float[] fxet = input.clone();
                float[] fxetb = new float[input.length];
    int size = input.length;
    int log = 2;
    float pi = (float) 3.1415926;
    //这个for for循环init可以拿出去new， 提高实时计算效率
    float [][]cosNT = new float[input.length][input.length * 2];
                for(int i = 0; i < input.length; i++) {
                        for(int j = 2; j < input.length * 2; j *= 2) {
                                cosNT[i][j] = (float) Math.cos((i - 1) * pi / j);
                        }
                }
        float Et_cosk_logn_bit_sign[] = new float [size];
        float Et_cosk_logd_bit_sign[] = new float [size];
        for(int i = 0; i < fxet.length; i++) {
                for(int j = 0; j < fxet.length; j++) {
                        Et_cosk_logn_bit_sign[i] = 1;
                        Et_cosk_logd_bit_sign[i] = 1;
          }
    }
        float detaT[][] = new float[size][size];
        float detaDT[][] = new float[size][size];
    //log - section - loop
    while(log < size) {
        //loop
        double Ei = 0;
        int logd = log >> 1;
        int logi = log << 1;
        //double detaT = cosNT[0][log * 2]; // 1/16
   for(int j = 0; j < logd; j++) {
        //去掉时函数的2倍角代换进行内核化T熵查询即可，
        detaDT[j][log] = cosNT[j << 2][log << 1];
        detaT[j][log] = 1 +  detaDT[j][log];
        //因为我设计内核是从0开始，而这里幂计算是1开始对2的倍数熵，
        //所以大家用于各类具体实际工程记得去校准一下，
        //section - loop
        for(int i = 0; i + log < size; i = i + log) {
    //Et_cosk_logn_bit_sign[i + j] = (float) (Et_cosk_logn_bit_sign[i + j] * detaT);
```

```
    //Et_cosk_logd_bit_sign[i + j] = (float) (Et_cosk_logd_bit_sign[i + j] * (1 + detaT));
    //于是跟进思考，这里的多倍区间熵减是一种交换律形态，不是单倍的detat 伽马集需要稍后计算讨论，但这种优化方式直接将熵成为了定值，o（nlogn） fft计算量因为定值提出去，成为了 o（n）也即是1024次。。
    // Et_cosk_logn_bit_sign[i + j + logd] -= detaT;
                }
            }
            log = logi;
        }

        //log - section - loop
        while(log < size) {
            //loop
                double Ei = 0;
                int logd = log >> 1;
            int logi = log << 1;
            //double detaT = cosNT[0][log * 2]; // 1/16
            for(int j = 0; j < logd; j++) {
             //fxet[j] *= detaT[j][log];
             double detaTi = cosNT[j << 2][log << 1];
                //section - loop
                for(int i = 0; i + log < size; i = i + log) {
                        //fxet[i + j + logd] -= fxet[i + j] * detaDT[j][log];
                        //fxet[i + j] *= detaT[j][log];
                        //Ei = fxet[i + j] * detaDT[j][log];
                        Ei = fxet[i + j] * detaTi;
                        fxet[i + j] += Ei;
                        fxet[i + j + logd] -= Ei;
                }
            }
                log = logi;
        }
        //fx 是当前的fxet子集对应的倍频率域 的带有符号和, 之后统一下类型
        for(int i = 0; i < fxet.length; i++) {
            //还没完，稍后。我要吃饭睡觉玩，今天就到这。
                fxet[i] = (float) Math.sqrt(fxet[i] * fxet[i] + fxetb[i] * fxetb[i]);
        }
        return fxet;
    }

//刚刚又思考了2个小时，关于log熵内积去时函数特殊关系和2倍角关系进行定值k提取，现在通过迪摩根的比特位计算进行归一化，我得到一个最简单的方式，当时没有考虑运动者的变化状态，只考虑了观测者的内积变化关系，于是现在观测者一旦分离，便将运动者的变化状态做个5000次简单的加法观测，看看效果。不做乘积变化，因为上两个版本就是5000次乘加。
                public float[] doSigmaLogEt2Logdfft(float []input) {
                                float[] fxet = input.clone();
                                float[] fxetb = new float[input.length];
                        int size = input.length;
                        int log = 2;
                        float pi = (float) 3.1415926;
```

```
//这个for for循环init可以拿出去new， 提高实时计算效率
    float [][]cosNT = new float[input.length][input.length * 2];
    float [][]sinNT = new float[input.length][input.length * 2];
            for(int i = 0; i < input.length; i++) {
                    for(int j = 2; j < input.length * 2; j*=2) {
                            cosNT[i][j] = (float) Math.cos((i - 1) * pi / j);
                            sinNT[i][j] = (float) Math.sin((i - 1) * pi / j);
                    }
            }
            float detaT[][] = new float[size][size];
            float detaTs[][] = new float[size][size];
            float detaET[][] = new float[size][size];
            float detaETs[][] = new float[size][size];
   //log - section - loop
   while(log < size) {
   //loop
   double Ei = 0;
   int logd = log >> 1;
   int logi = log << 1;
   //double detaT = cosNT[0][log * 2]; // 1/16
   for(int j = 0; j < logd; j++ ) {
    //去掉时函数的2倍角代换进行内核化T熵查询即可，
    //double detaT = cosNT[j << 2][log << 1];
    detaT[j][log]= cosNT[j << 2][log << 1];
    detaTs[j][log]= sinNT[j << 2][log << 1];
    detaET[j][log]= 1 + detaT[j][log];
    detaETs[j][log]= 1 - detaT[j][log];
    //因为我设计内核是从0开始，而这里幂计算是1开始对2的倍数熵，
    //所以大家用于各类具体实际工程记得去校准一下，
       //section - loop
   }
    log = logi;
}
log = 2;
float[] dlt = new float[input.length];
float[] dlts = new float[input.length];
for(int i = 0; i < fxet.length; i++) {
    dlt[i] = 1;
    dlts[i] = 1;
}
while(log < size) {
   //loop
    double Ei = 0;
    double Eis = 0;
    int logd = log >> 1;
   int logi = log << 1;
   for(int j = 0; j < logd; j++) {
      for(int i = 0; i + log < size; i = i + log) {
            Ei = dlt[i + j] * detaT[j][log];
```

```
                        Eis = dlts[i + j] * detaTs[j][log];
                        dlt[i + j] += Ei;
                        dlt[i + j + logd] -= Eis;
                        //因为减熵是不规则的补码加乘逻辑，于是剔除，
                        //因为剔除了蝶形的倍频减熵，于是用sin来补充迪摩根互补律。
                        dlts[i + j] -= Eis;
                        dlts[i + j + logd] += Ei;
                            }
                    }
                    log = logi;
                }
        float En=0;
        for(int i = 0; i < fxet.length; i++) {
          //加sigma 加 分数做了几组简单测试，可以跟进，这里略
              En += fxet[i];
        }
    //fx 是当前的fxet子集对应的倍频率域 的带有符号和, 之后统一下类型
        for(int i = 0; i < fxet.length-1; i++) {
    //1024 * 5 的乘法计算次数
    //fxet[i] =  (float) Math.abs((fxet[i] * dlt[i] + fxet[i + 1] * dlts[i])*(fxet[i] * dlts[i + 1] + fxet[i + 1] * dlt[i + 1]));
    //1024 * 4 的乘法计算次数
    //fxet[i] =  (float) Math.abs((fxet[i] * dlt[i] + fxet[i + 1] * dlts[i])+(fxet[i] * dlts[i + 1] + fxet[i + 1] * dlt[i + 1]));
    //1024 * 3 的乘法计算次数
      fxet[i] =  (float) Math.abs((fxet[i] * dlt[i] + fxet[i + 1] * dlts[i])*(dlts[i + 1] + dlt[i + 1]));
    //1024 * 2 的乘法计算次数
    //fxet[i] =  (float) Math.abs(fxet[i] * dlt[i] + fxet[i + 1] * dlts[i]);
    //1024 * 1 的乘法计算次数
    //fxet[i] =  (float) Math.abs(fxet[i] * dlt[i]);
        }
        return fxet;
            }

      //内核定值优化版本 基于FFT进行去递归，
            public float[] doEt2LdfftOpt(float []input) {
                    float[] fxet = input.clone();
                    //float[] fxets = new float[input.length];
        int size = input.length;
        int log = 2;
        float pi = (float) 3.1415926;
        //这个for for循环init可以拿出去new， 提高实时计算效率
            double [][]cosNT = new double[input.length + 1][input.length * 2 + 1];
            double [][]sinNT = new double[input.length + 1][input.length * 2 + 1];
            double [][]detaT = new double[input.length + 1][input.length * 2 + 1];
            double [][]detaTs = new double[input.length + 1][input.length * 2 + 1];
            double [][]detaT2 = new double[input.length + 1][input.length * 2 + 1];
            double [][]detaTs2 = new double[input.length + 1][input.length * 2 + 1];
                    for(int i = 0; i < input.length; i++) {
                            for(int j = 2; j < input.length * 2; j*=2) {
                                    cosNT[i][j] = Math.cos(i * pi / j);
```

```
                                sinNT[i][j] = Math.cos(i * pi / j);
                            }
                        }
                        for(int i = 0; i < input.length; i++) {
                            for(int j = 2; j < input.length * 2; j*=2) {
                                //cosNT[i][j] = Math.cos((i - 1) * pi / j);
                                if(i <= input.length/4 && j <= input.length/2) {
                                    detaT[i][j] = cosNT[i << 2][j << 1];
                                    detaTs[i][j] = sinNT[i << 2][j << 1];
                                    detaT2[i][j] = cosNT[i << 1][j << 0];
                                    detaTs2[i][j] = sinNT[i << 1][j << 0];
                                }
                            }
                        }
                //log - section - loop
                while(log < size) {
                   //loop
                    double Ei = 0;
                    int logd = log >> 1;
                    int logi = log << 1;
                    for(int j = 0; j < logd; j++) {
                       //section - loop
                       for(int i = j; i + log < size; i = i + log) {
                            Ei = fxet[i] * detaT[j][log];
                            //Eis = fxet[i] * detaTs[j][log];
                            fxet[i] += Ei;
                            //fxets[i] += Eis;
                            fxet[i + logd] -= Ei;
                            //fxets[i + logd] -= Eis;
                          }
                   }
                    log = logi;
                }
                for(int i = 0; i < fxet.length; i++) {
                    fxet[i] = Math.abs(fxet[i]);

                }
                return fxet;
        }
}
//1/8 -- 2/16 --   2 * detaT[] * detaT[] - 1
//2/8 -- 4/16 --  2 * (2 * detaT[] * detaT[] - 1)
//3/8 -- 6/16 --  3 * (2 * detaT[] * detaT[] - 1)
//4/8 --
//currentC = 2 * detaT * detaT - 1; // cosNT[0][log * 2]
//currentC = 2 * j * detaT;
//currentC = cosNT[2 * j][log * 2];
//还可以用时函数跟进优化，2028年再公开。，
//currentC = regc;
```

//这种只考虑 +- 2cosAcosB +- 1 中关于的 + 2cosAcosB - 1 时候 A 又= B 的 绝对固定的2倍角公式条件来化解分数熵对数用法，有效计算小于2万次，但比之前除以2的熵化7000次要多，质量还保真。
//我罗瑶光给大家的提的疑问是为什么不用同理的sin变换来处理fft倍频熵？我定义为大三或者研一的难度。

关于log熵化公式 优化时发现的一些算法逻辑归纳

1 log的倍数幂数和差化积 模型

```
/*
 * log 熵化 关于2的指数倍数幂 和化积公式  Log(A + B) = 2 * LogA * LogB + DetaC
 * 当A = B 时候， DetaC= - 4 * E（n-2）-（n-2）
 * 发现者 罗瑶光- 430181198505250014
 * 应用1 离散傅里叶On核 倍角聚变 实现 。 范畴1 属于时函数的新增公式集。
 * */
```

2024年时函数个人英文笔记5篇

Benefits of VPCS in personal opinion, to create a sound business environment.

Due to the time norms could walk from a computational ware-base to a science engineering, hence the VPCS also should get an ability, to walk from the ware-based component to the field of economics and managements. At almost universal truth that kinds of classical theory, commonly be a macroscopic system, such as ecological flourished-relationship, VPCS needs expecting more and challenging arrangements with global industries. Either rise up the efficiency of man-hour, or enhance the vocational vacations.

Yaoguang.Luo --Liu Yang city --20240730

Cohered a fractorial gradient with differential time norms.

Lapsed a poi-lion's decision in conclusion-dispersions. Scientist rarely railroaded algebras to enhance the archives in their statement of working papers. Obviously compounded exponential clothes by E estimates with fractorial and polynomial theory. Seems preciously wasted times and stationeries. Time norms vigorously decomposed a binomial equations into the atonical processions and observational dismantling, instead of symbolical wedges to demonstrate the benefits of computings. Passionately engaged the computations frequently, at mean time involved conscious and literal utilities to enrich the depitional cerebrations orbitally.

Yaoguangluo --Liuyang --20240518

Ironic Horoscope of Time Norms

DimorganS rotation in adaption of statistics, could be summarized from a fractal inner product to an idempotent outer product. Time norm calculus rapidly decided the dos transform of topological convolutions. Similalized a meadian-calculation with Hilbert's logarithm of destrictional room's clusters. Which unoverduring rotatly between overlaies and deltas. As a without scaled sigma's dynasty. Sustainably promoted a separation of Dimorgan's descreted sets, and

extensively appeared in applicational domain of Descarte's combinational disorders. Or other kinds of distributional algebras and dualsm of diminished quantums.
f.xy = (l.f.x + l.f.y)* l.t.n + l.t.c.
f.x + f.y = l.f.(x + y) * l.t.n + l.t.c.
Yaoguangluo --Liuyang --20240515

Cutting edge. The sheer amplitude of timer norms.

Passed canonical algorithm by a

conventional computing. An eyesight under latest incarnation of timer norms where vias an abbreviation of sufficient condition. Neats encapsulations of optimical and versatical characters. Yaoguang.luo considers an advanced quality wareutil, which fits a genre computing server. It is obligatory in plural domain of calculus, statistics, probabilities, geometry and at least of quantum. Seems a morphology of evolution to innovate the patternal transcomputing, which will come true.

Yaoguangluo --Liuyang --20240512

Fractional computing and inferential meterings

A charm of consolidations and cerebrations, in a wise of prophet that prolonged a logical consciousment. Absolutely it incarnational remedy in different domains of topology, perceives, spatial and imaginal circustances.
Recently it almost be confined by multi corporations in gaze of two perspectives.

1 Digging assets from a vague, chronic envisages about the environment. By using several technologies such way of statistics and Math. 2 Definitely explicit the sectional propulsion to make a segment of elevate the results with algorithmic nominations.

The way of clench the rendering of obligations in computing servers, notably stimulate a vast meticulous scalas, Mr Yaoguang.Luo listed two achieves as below. Timer norm the cardinal and differential functions and DNA INITON encoding Dics. which could ingeniously vessels a prevalent wisdom good here.

Yaoguangluo --Liuyang --20240510

时微分函数归纳

关于时间函数，首先我的所有的数据，我都是通过微分和积分求导来计算出来的一种方程。目前这个方程在养疗经的软件中有实际的应用关于噪音的识别和过滤。我感觉有必要申请著作权。于是准备将这些文字进行一种文章的形式来进行记录。国外通过GitHub，国内通过知乎进行备份。

在记录这个文章之前呢，我感谢我已经有了著作权形式的时间函数的架构，然后关于这个架构到最后的时间函数的方程解析，我会用文字的形式，在这个文件夹下进行详细的描述。因为时间函数是通过三维的这个世界的数学观来进行解析时间函数的模型，所以在很多领域进行应用的时候呢时函数模型里面的符号并不等同于三维世界的数学符号，所以首先在这个序言里面呢，我会将符号进行严谨地定义和标识，在这篇序言里面先包含两个部分第一个部分就是我得到的时函数的公式集，第二个部分是时函数公式的符号的定义标识。

第一个部分就是我得到的时函数的公式集，

|tero(x)> + |tcol(x)> = deta (t1-t0) * m(t) / deta1(t1 - t0) * p(t) = 1 （感谢这个母公式）

|tero(x)> + |tcol(x)> = deta.et = 1

f(|tero(x)> * |tcol(x)>) = 0 + C

公式对应工程物理和数学的逻辑解析

E = [f(d)] * [f(d)] * i * di（爱因斯坦能量公式，与时函数间接关系，本文可剔除，本人利用这个公式用fft的振幅能量转换，发现时函数在fft计算中通用）

deta.it = 1/ (2 * sqrt(i)) * sin(A - B)

势能与虚空的关系

ft + fd * fr = E

fi = ft * fr / fd

ft = fi * fd / fr

质量能级与空间变换的噪音计算方程

l.f.deta.n = 1/N - 2 * cosA * cosB

单位时间粒子与质量的关系

fd = {- sin(A + B) * (2 * cosAcosB - 1)}^(1/2)

能量在时间与空间上的表达关系

fi = cosAcosB * (cosAcosB - 1) + sinAsinB

fio= 1/ (2 * sqrt(i)) * { sin(A + B)sin(A - B) - 2 * sin(A + B)sin(A - B) * cosAcosB }

时间与能的表达式

te-> = f（i * di）+ c = sin（ai - b）** en *（t * 1 / sqrt(i)）** en

时间微分方程

ff（deta.e * deta.ft）=f‘ * (mm * mv) / (ms * mt) = e->dt

时间微分与复变域的关系

deta ik = deta（a - b）/deta1（a - b）= cos.a * cos.a * cos.b * cos.b - cos(a + b) + c

时函数能级的模

f.deta.et-> = A(A +- 1) +- B

B = sin.a * sin.a

C = sin.b * sin.b

A = B * C

虚空与内积的关系

f.k = f.deta.ti' * f.deta.d'/f.deta.r' + f.deta.d' * f.deta.d' = f.deta.ik * (1 / 2 * sqrt(i)) + C

虚空展开式

f（a + b * c- e / f 。。）= f.deta.a + f.deta.b * f.deta.c 。。。。。 = f.deta.a-> + f.deta.b-> + f.deta.c-> - f.deta.e-> - f.deta.f->.....

爱因斯坦流逝时间deta.t 与 罗瑶光时函数流逝比重 deta 关系

deta.t = 1 / deta = 1 / （cos.a * cos.a * cos.b * cos.b + cos(a + b)） + C = deta1(a - b)/ deta(a - b)

上面公式的理解力涉及的方程基础常识
1 工程数学
2 离散数学
3 微分拓扑学
4 高中代数 上下
5 物理学

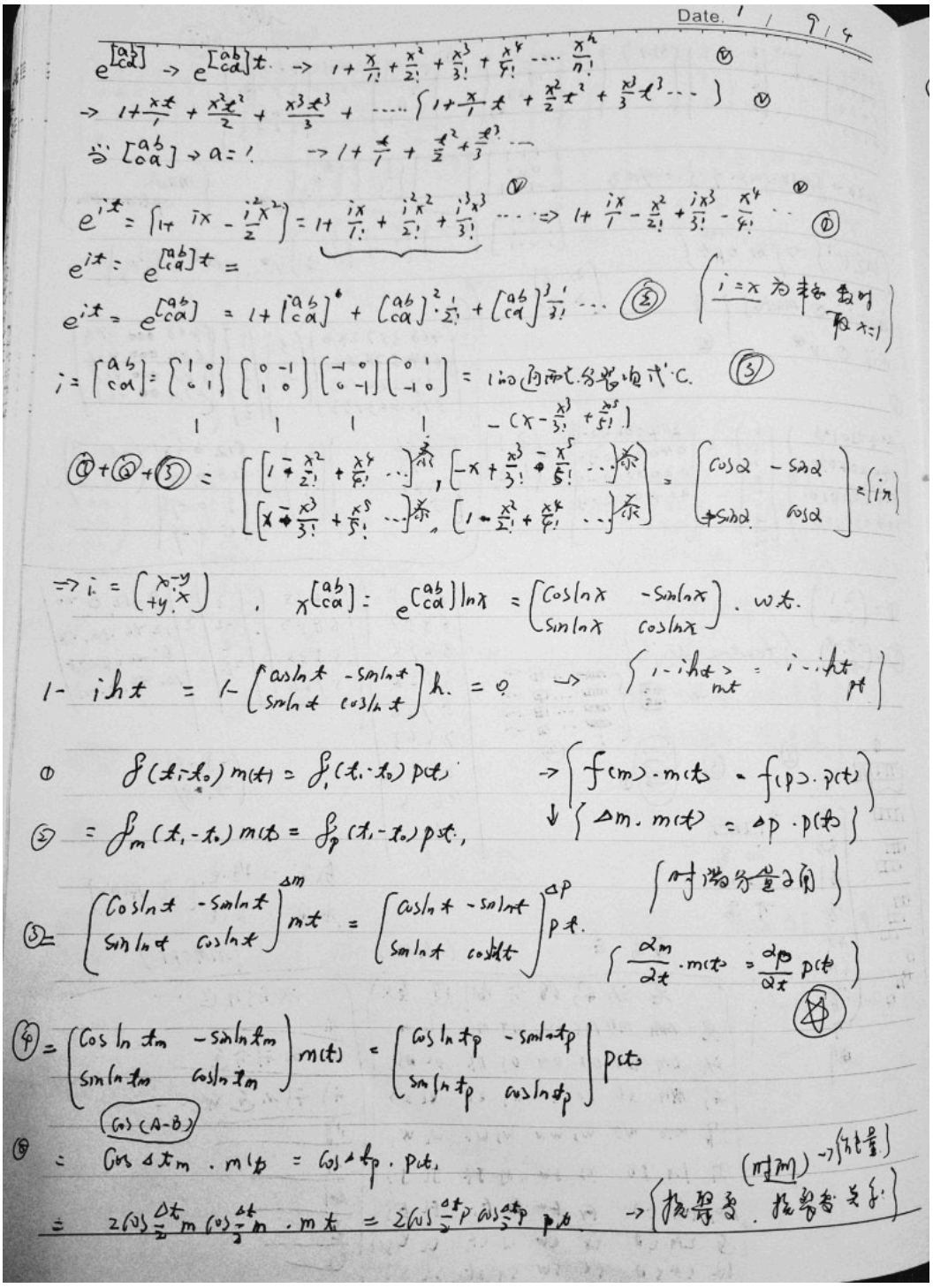
下面符号的理解力涉及的方程基础常识
6 解析几何
7 量子数学
8 光学
9 数字逻辑

第二个部分是时函数公式的符号的定义标识
线性偏移关系 + 罗瑶光先生认为+-不应该是数学与思维范畴，包含时空的偏移，环境的变换等模型。
线性偏移关系 -
自由线性偏移关系 +-
曲线偏移联系 *
曲线偏移联系 /
右上特殊状态标记 ** 指数与对数 也属于一种数学标记，
扩号包含关系 .
常数 c
时间 t
空间 d
量化 q
运动 p
能量 e
物质 m
物质的观测 mm
活动的观测 mv 如空间分布
活动的状态 ms 如牛顿定律
存在的时间 mt
虚空 i 有必要解释下虚空的感念稍后。
内积 k
距离 s
有效域 deta 有必要解释下有效域的感念稍后。
过程 xn
导 '
积 f
无理级系数 l 有必要解释下
态 ->
函数 f(x)

最近用虚空时在多个笛卡尔计算(dnn内积，索贝尔卷积，迪摩根条件，非对称切分)领域广泛实验，全部论证有效通用。并归纳了一组新的变量

正弦符号 ---------- 事物的操作变化------OED

正弦符号的X轴对称符号 ---- 事物的维度变化------PCU
w符号 ---------- 事物的时间变化------AIV
无穷大符号 ---------- 事物的消逝过程-----MSQ
右上箭头符号 ---------- 事物内部的变化
左下箭头符号 ---------- 事物外部的变化
圈内加符号 ---------- 事物内部的关系
圈内减符号 ---------- 事物外部的关系
圈内点符号 ---------- 事物的关联
{} ---------- 一组参照物
N ---------- 时间熵
其他和微积分类似略
时函数1-iht 在凯莱 牛顿 郎洛 视野中的理解方式。 i 与 detα T的关系



之前在思考薛定谔层面 1-iht | m> 与 1-iht | p> 可优化方式，上图是其优化得到斯塔等式的全过程。通过凯莱线性矩阵加上泰勒展开与三维视角组合，再通过欧拉分解来进行凑方法论，可以得到朗洛层面的斯塔等式。因为矩阵的和差化积可以2倍角替换，那么单一一个t1-t0的德塔变化量就可以概括基于这个观测点的全部观测量。

图中难以理解的只有关于矩阵的cos linT 和 sin linT的 四个算子，这里描述下，这类算子再明确了-1 到 1 区间变化，那么相当于cos确定振幅，那么 cos里面的k pi 变化分量只是时间的变化过程描述，于是关键参数锁定在了mt和pt的方式上。

关键词解释- 凯莱 矩阵--涉及三角函数的矩阵解析 - 微分拓扑学课程

泰勒展开-涉及柏松分布 和 泡利展开式解析 - 印度基督大学课程

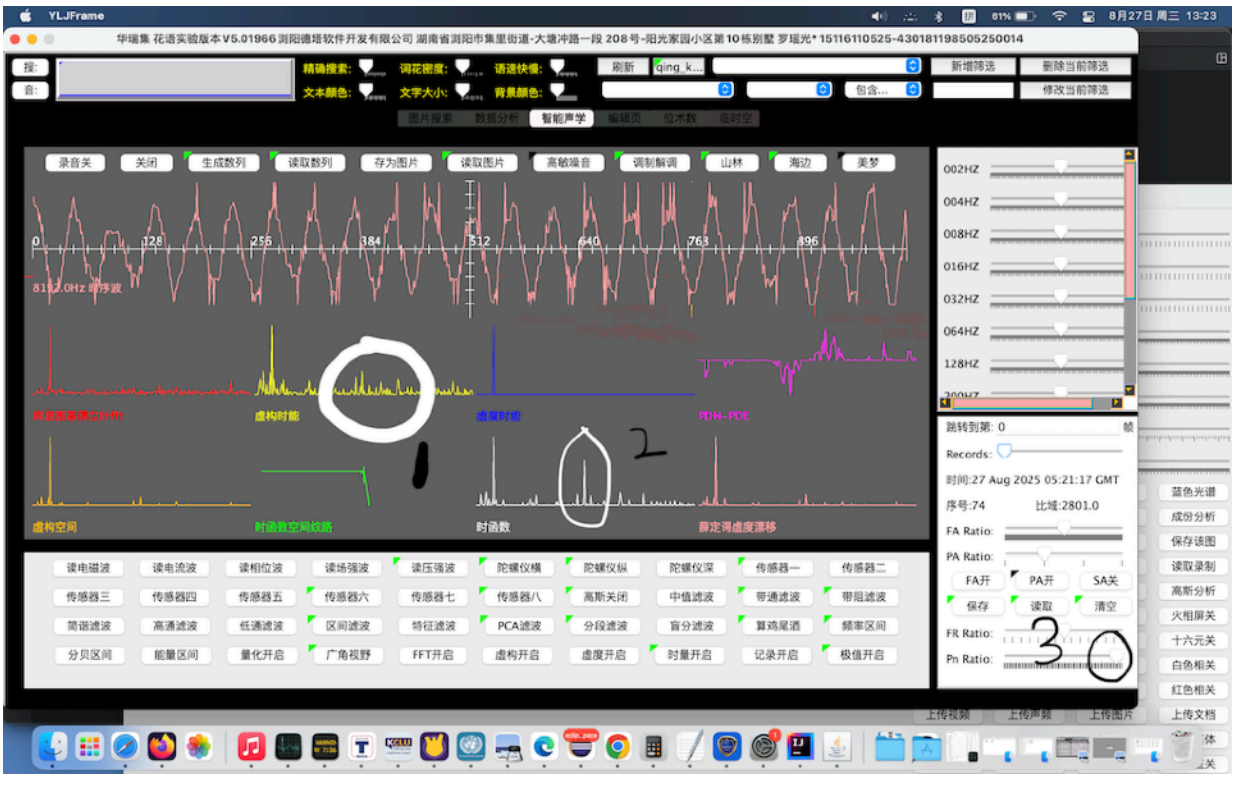
和差化积-高中知识

肽展公式滤波变换

声音处理中的细节进行描述图中最长的淡红色波形为真实的声音进行录制和解调观测, 下面的8个小图是在傅立叶变换后的频率波形观测方式.

可以发现左上角红色的FFT频率为一个大体单帧化波形, 不能很好地观测噪音频率帧, 于是集成罗瑶光先生的时函数进行仿真细化fft的碟形空间, 价值在主峰帧可以1+- 2cosAcosB的区间范畴拓扑分解和叠加, 用一种势能转化动能的方式分解, 然后动能碎片再拓扑成具体点的势能叠加, 类似雨滴在湖表面的泛化过程. 产生观测可读模型.
体现在圈 1 处 -很好地处理一些频率区间所产生的对称的傅立叶2 K pi 角分解成不规则的角度拓扑微分.
因为圈1是一个宽域分贝通量, 不能准确的定义精确频率峰位置, 于是在通过十六元基肽展公式进行PDS PDE变换增加酸碱精度调度, 一些平滑的曲面上的产生的微小凹凸能够进行强烈的极值域拉大, 于是波形中一些微弱的高频的震动变化能够被剧烈放大, 如圈2 的 峰帧就产生了.

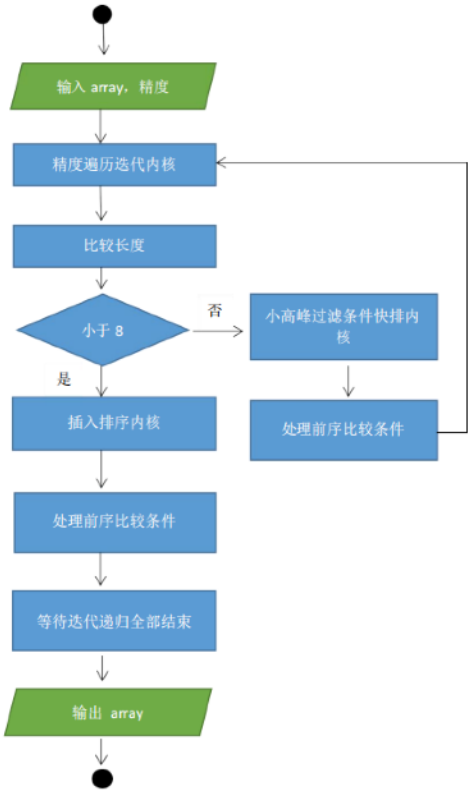
图中圈3时肽展公式酸碱模拟操作应用. 这个应用作为论据可以很好地证明肽展公式不但之前在图片上可以处理色阶快速分析和观测, 在波形上也有同等效用.



计算的模式变换

1 价值应用

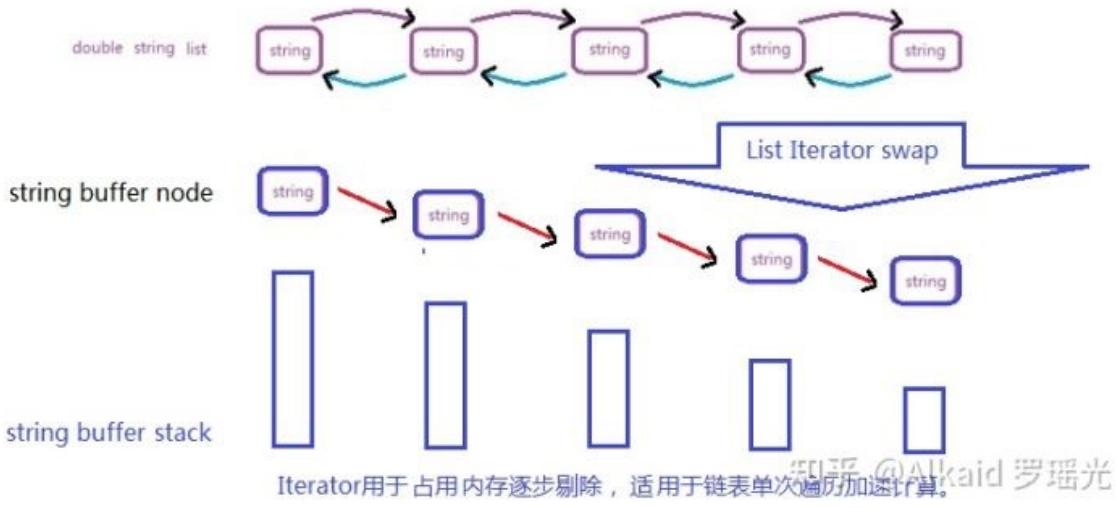
德塔数据结构计算的模式变换主要用于 buffer中间态变换. refer page, 如混合数列排序的变化



作者的Flow Chat 画图基础来自印度基督大学的MCU at 8085. 导师 JP Sir.

2 buffer中间态

包含 map与tree的变换, list与iterator的变换. refer page 520, 537, 516, 515



图中String buffer stack 可先后序列排列, 可断开成链, 高度是iterator对象当前的内存占用大小. 罗瑶光画图

list. toIterator()变换模式优势罗瑶光先生个人认为在计算过程中, 基于内存的占用和寻址效率加速. String to Stringbuilder 变换同理, 对象buffer化能实现内存变量计算和调用进行极限加速. 作者在印度基督大学 学数据结构没有stringbuilder和 iterator的知识点, 在2016年亚马逊的岗位技术经理面试时候, 有几次印度经理多次面试我关于String计算方式, 我当时没有答上细节, 错失了月薪12000美金的工作. 我的罗瑶光画图

3 模式变换计算趋势归纳 --稍后

主要为非线性与线性的降维变换, 通过改变观测面实现. refer page 497

应用



罗瑶光画图

一些格式数据在变换中的细节引起的思考

1 罗瑶光先生在处理波形调度的过程中，多次遇到数据解调失真，于是开始思考大学2年级的问题-统计学-，如何有效规避这类问题。

2 在设计时函数的时候采用了简单的保真策略，采用了剔除法满足高速高质的观测输出，于是开始思考大学3年级的问题 -偏微分-如果 在极不稳定的场合，剔除法如何优化？

3 在设计优化场和坐标轨迹环路的时候，于是开始思考大学4年级和研究生1年级的问题，如何更加精准快速地计算空间中主要坐标团重心区域切分，

4 上述三个问题组合成一套 -微分几何- 笔记， 解决了罗瑶光在家身边的很多真实问题，于是变成思考题在这里活跃下大家的思维。

第六章_数据预测引擎系统

基础应用: 元基催化与肽计算 编译机的进制仿生计算机

知识来源,

作者人生第一次对坐标计算产生概念理论思维,来自印度基督大学的Dr Fr Joseph Varghese. 作者基于离散数学第六版紫皮书,完成了所有课后习题和记录Varghese留在黑板的所有课堂笔记包含完整的神经网络和坐标思维,曲线思维的欧式计算基础.

作者的坐标计算思维活力来自游戏设计娱乐,如作者的暗夜法师object C设计的游坐标战斗游戏,这里感谢下路德大学卡拉森教授,关于作者设计她的数据挖掘分析课程,作业检查极其严谨,如knn基础,欧拉,欧基里德等 欧式家族权径作业.

作者曾经在章总那关于co-auther图设计用了2维的vector,阅读了很多美国亚利桑那大学的co-worker图模型文章论文,这些基础,得益于作者在国内九年义务教育和高中以上学历教育的良好数学功底(作者高三虽少学了半年课,但浏阳一中此时正在复习高二的立体几何1年,写这本书,作者综合还是赚了,作者有这个技术实力,基础离不开浏阳三中罗佛成老师).

最后作者在设计养疗经智能相诊, 三维词汇花工作中,业余挑战leecode, coderank题目,以及生活中的问题处理,积累了大量数据计算方法的经验,于是开始了数据坐标计算的插件包设计. (作者在长城驾校学重驾, 知识点是考侧方位停车, 单边桥, 压饼, 上坡定点停车, s路, u型倒车, 雨天驾驶, 十字路口通行, 雨雾天行驶,闹市行驶,障碍避险,山林隧道驾驶灯光模拟等,里面也有基于东风货车的后视镜,车灯,车门把手, 车护, 车头线的对点对线对角对表的思维. 但不包含坐标群的轨迹概率路径分类聚类计算. 这里申明. 这里还是要感谢下长沙驾考中心, 当年200元半小时场路训练, 作者亲身经历了花最少钱测试最多的训练科目的挑战,作者把科目的地点当成坐标,怎么进行欧拉环路计算最短时间路线做最多测试哈哈)

关于雷达工学描述: 作者的教研室主任韩桂明先生是 物理专家,是作者的技术导师,虽然作者的毕业设计是文学论文,但是很多计算机物理的基础来自于韩桂明老师的教学,虽然作者在本科的专业没有涉及雷达研究课程,但是关于电工学基本元器件的启蒙,如物理电工, 三项原理, 触发器, 555定时器等知识学的比较扎实. 以及后来作者在上海学了下的Atten2000 等示波器 微焊 以前有一篇文章描述过他的记忆. 韩教授是军方人员，避免误区，于是多点文字描述下，我和韩教授私下见面其实就2次，一在教室，我说你还记得我吗，上次爱因斯坦书被你收了。一次说我还想再读一年大四。平时他的课我都是不及格的。降分及格线我都达不到的。幸好我不逃课。

Story: Professor HanGuiMing was a good teacher, all year round in the radar field research, he was my engineering department director, he taught me the computer and the physics, I still remembered that one of the sophomore physics classes, really bored I were at that time, I sneaked <<Einstein's famous quotations with English style>> out from the bag and watched it. I' m sorry that Mr. Han was finding what I did. He took my book away and exclaimed:” well!!!, when I was a kid, I wanted to be Einstein, but now the result that I failed. hey~ Mr. Luo, I hope you can do”, I ashamed and face turned red, embarrassed suddenly. Since then I had never deserted, and also from that time, the teacher gave me the particularly "care", especially when the MCU course in the school, Mr. Han selflessly to give me many of his physical micro processor records. Professor Han was my unforgettable teacher. College examination was very difficult. Thanks for the N.A.A School, 4 years of the sculpture care for me. 罗瑶光

api包 函数完整包含2维和3维的空间轨迹算法.

GitHub - yaoguangluo/Data_Prediction: 快速计算商旅轨迹 非线性坐标数据分析

Java api https://github. com/yaoguangluo/ChromosomeDNA/blob/main/BloomChromosome_V19001_20220108. jar

降维计算 坐标系统预测

1 数据预测引擎的坐标系统主要用来做离散非线性计算.

refer page 566~ 在面非线形的对图计算，像素点可以坐标化标记，这是一种可行的细化过程。

2 离散非线性计算主要体现在 降维 商旅TSP路径的线性求解.

refer page 629~ 坐标太多的时候可以进行指数 含噪簇化节肢进行含噪计算，避免耗时间的精确叶计算。

3 坐标的降维计算包含 轨迹降维, 趋势降维, 观测降维.

refer page 567~

4 降维计算过程可以进行逆向跟踪还原.

refer page 570

关于降维计算的价值主要体现在解决指数计算带来的的巨数问题，通过模拟拆解log的方程形式将大指数进行迭代缩小，举个例子2的 5字方 和 2的50字方，计算量天壤之别， 一旦进行递归分层和逻辑迭代，那么变成 5 * 5 * 2，总体区间化降维成2的5字方 * 10个组 计算量最终可优化成 2 的 9 字方。

运动趋势 环境预测

1 数据预测引擎的环境计算主要体现在 压力计算.

refer page 570, 573 --压力的来源属于一种力学的拓扑，这个过程可以产生很多可观测的面，点向的统计力便是一种。

2 压力计算可理解为 中心向重心的两点间距离.

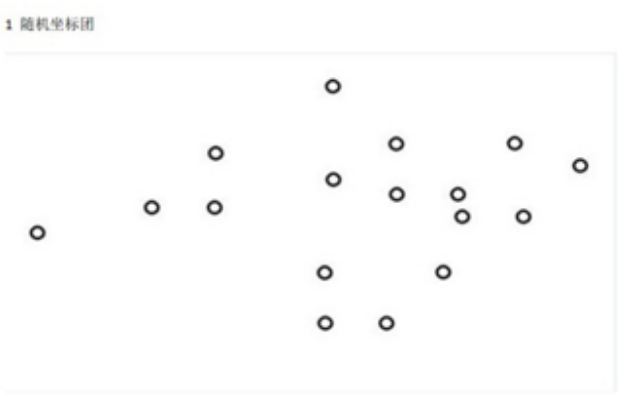
refer page 674 --压力的统计对象点集合 进行邻近的，逐步的和 由下而上的 聚类叠加距离值域可以观测其不饱和状态的变化趋势。

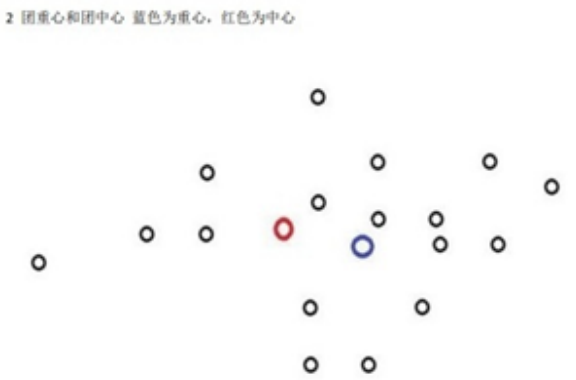
3 两点间距离的长短和方向代表压力的大小和趋势.

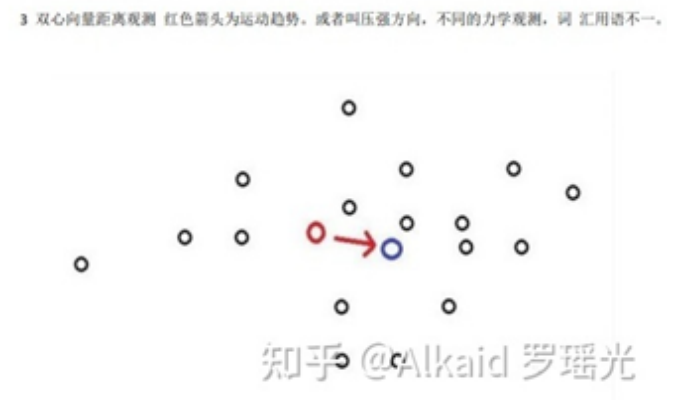
refer page 574 红蓝点距离 --这些趋势一旦向四周向量散射通过四则运算和角分解发现偏差，这个偏差可以理解为运动势能，一旦产生位移，可以理解为 势能向动能的转换过程。

4 趋势大小确定环境的稳定性表达.

refer page 574 --当然势能计算统计的均值总是趋近于平衡，可以理解为所在的空间能域稳定。热变化率低。







边缘探测 雷达机

1 数据预测引擎的雷达机主要体现在坐标群的边缘识别和归纳计算.

refer page 577 --边缘可以确定积计算 ， 用途广泛。

2 坐标群的边缘识别和归纳计算 采用角度 + 中心到点距离进行进行轮循链接.

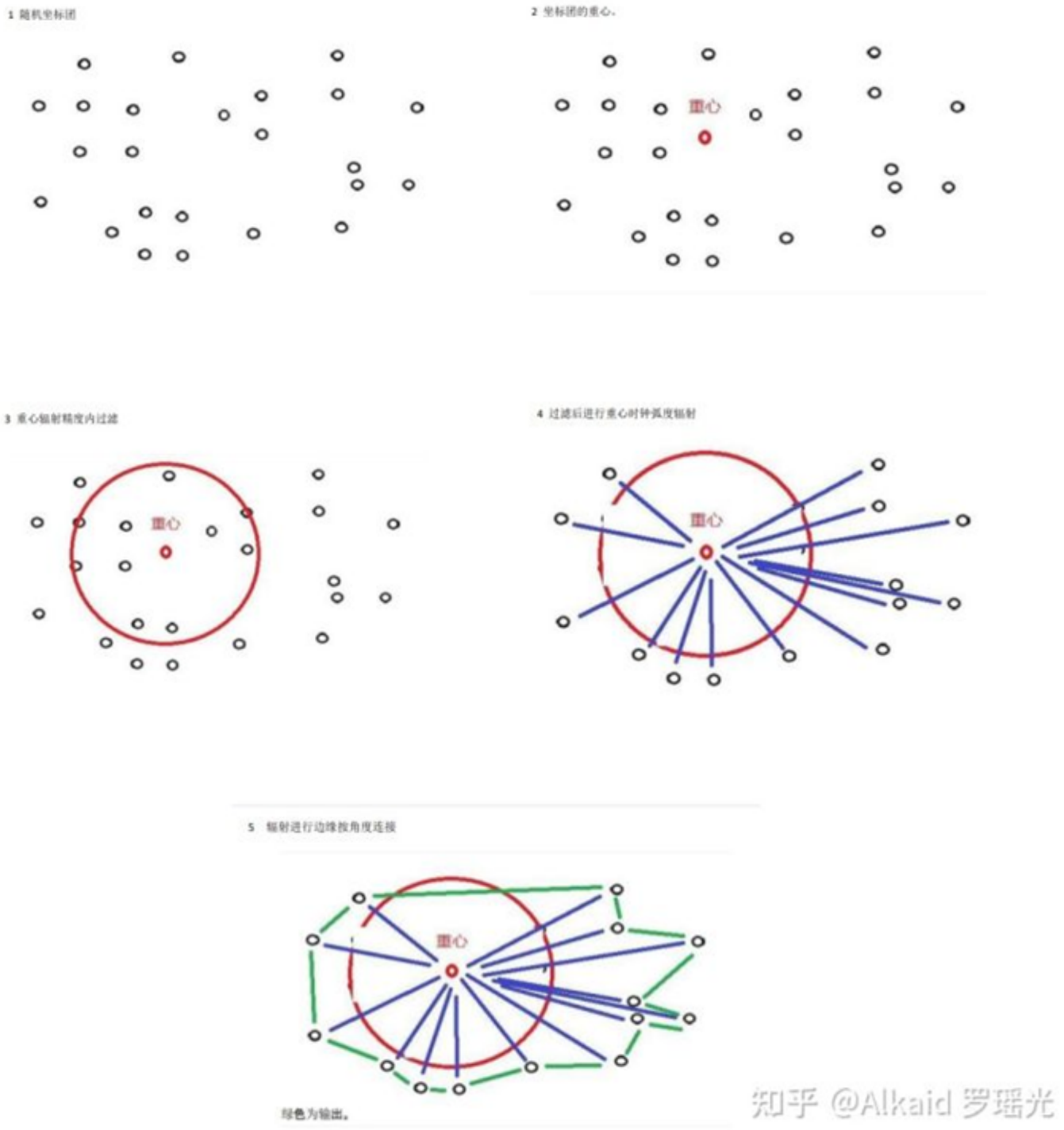
refer page 576 --角度比值相同的标记，仅保留最大位移距即可。

3 链接的面形成 极速计算边缘包含, 确定坐标的团大小面积, 密度.

refer page 577

4 极速边缘计算的价值可以迅速利用在所有实时坐标系统中.

refer page 593



整理漂亮些, 迫使作者设计这个算法, 最早来自作者的计算机视觉课程关于Reinhart给作者布置作业求坐标的边缘链接算法, 作者当时用的是谷歌搜索的传统经典的切线向量计算, 基于一个个坐标来解边缘的坐标链接, 作者一直在思考切线角度算法, 速度太慢了. 跟作者第一代TSP超过23个坐标点后的计算速度有的一拼, 坐标点越多, 就越慢. 这个算法问世后拔去了作者至少5年的心头刺.

Radar Mechine

The Data Prediction of Side End Radar which is used for the recognition of edging coordinates. The computing of generalization is circularly to find a ratio collection of vectors between the central weight point and each outer coordinates. Compare to the outer coordinates, because the distance from inner coordinates which is shorter than distance of scale input, hence will be removed by this scale filter. Then makes an arrangement of this vector list more sequence,

means sorting them from the ratio of zero to the ratio of 360 (2* Pi). Finally makes the line connections of each two outer adjacent coordinates in this vectorial list. It may get a side end of input coordinates. Then find a density is simply, because the author considers: {Density = Area of Side End / Amount of Coordinates}

Author YaoguangLuo 稍后优化语法

Gitee 20200306 感想, 2011年我在做路德大学计算机视觉的项目时候, 有一个作业题是要我标记坐标边缘集合, 我当时是用斜率梯度与切线比来计算临近两个边缘坐标的小线段系数, 记得当时我只有20来个坐标计算, 得到结果花费了我几秒钟时间, 最近养生软件也遇到相似功能, 需求迫使我重新设计一种新的算法, 于是这个算法就设计出来了. 前后计算10000坐标也就花费了几秒, 速度快了600~3000倍, 于是我将它开源了.

今天来描述下这个函数的运算逻辑:
1需求, 写这个算法, 我的动机是出现不规则的坐标团, 我怎么快速的将团的边缘坐标集合拿出来. 2当时所想, 如果我将团的重心求出, 然后基于重心坐标辐射团的每一个子坐标, 一旦距离小于scale精度, 就过滤掉, 那么这个算法应该比传统的边缘计算速度要快数百甚至数万(指数方)倍, 于是基于这个逻辑开始写论证代码. 3效果, 效果稳定, 可以很好地观测边缘坐标分布. 缺点也有精度不高. 4结论, 精确度会出现点误差, 为了弥补, 我思考了很久, 我将这个算法定义为 重心距过滤边缘算法, 适用于海量坐标团的边缘计算中过滤非主要成份预处理, 减少计算总量, 提高算能. 有效高精度处理模式为: 先重心距过滤边缘算法, 然后斜率梯度与切线比边缘计算(其实还有更好的思想, 比如基于重心和过滤后的坐标距离 排序, 取最长的百分比坐标集, 因为这些坐标往往属于外围坐标.), 再输出. 6另外, getDistance函数 原理是维度坐标的 向量位移 绝对值总和, 在计算速度上更快. 如果小伙伴在自己的工程中需要高精度, 可以改写成 开方的 欧基里德位移即可.

能量分解 模拟状态机

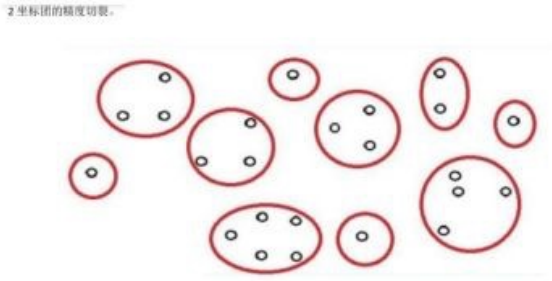
1 数据预测引擎的状态机主要包含 压力状态, 轨迹状态,

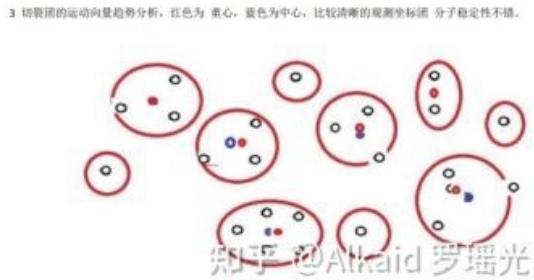
refer page 571, 573,

2 压力状态体现在坐标团的之间的距离, 和团中心和重心的距离分析.

refer page 571







Gitee 20200305 感想 《通过坐标团的 精度匹配分割的内部坐标聚类团 进行 每个聚类团的 重心和中心距离 求解 获取有效地团稳定观测数据模型》

关于这类算法, 早期思想, 我想设计出一种机器人的视觉观测算法, 能迅速有效通过坐标点集合计算出坐标团的运动轨迹. 用在运动趋势和轨迹判断等领域上. 因为作者开发条件有限, 无法从硬件和物理层来进行实验, 于是最早研发的主要软体观测数据是重心和中心这两个关键词. 一开始作者认为团坐标的中心如果和重心能吻合, 则表示观测对象团趋向于稳定, 于是从这个关键点入手. 发现一个豁然开朗的数据轨迹领域. 如果坐标团的重心和中心有位移段, 作者认为在某种观测角度上属于 势能, 这个势能 一旦解释为向量, 可以定义为运动趋势, 怎么计算这种运动趋势? 作者加入了稳定性, 向量和压强等参考名词来解释这个计算过程如下. 团坐标的中心 意思是团的中心点. 团坐标的重心 意思是团的PCA密度点.

如果中心和重心坐标临近, 则位移短, 作者定义为该坐标团稳定非充能状态. 如果中心和重心有位移, 那么从中心到重心的方向为物体的运动势能方向, 团趋向于该方向运动, 重心区的密度大小和重心中心位移同时确定了运动势能的大小. 如果将该重心区的团坐标集合再次切裂成小团, 计算每个小团的 运动趋势和轨迹, 进行微积分统计, 就能正确的分析出大团的整个运动周期过程. 之后的函数有相关基础编码.

动机, 早期动机为当坐标切裂融聚算法 完成后, 作者想基于此做工程实践, 确定它的使用价值, 于是想在交通领域上模拟, 后来发现这个思想可以在任何运动学和流体力学, 分子力学上施展, 于是有强烈的动机来完善它. ***去主观情绪***

当时所想, 这类函数的观测点毕竟只有3维, 重心, 坐标数和中心, 社会领域适用面已经无比宽广. 如果能再加一维 参数(比如体积等), 那么这类基础算法的社会应用价值将再次无限扩大.

Notes on Gitee 20200305, cords-clusters separation according to the fissile-scale were based on adjacent-distance of each two cords, then found each trend-distance between Its cluster-centre and cluster-weight. In order to make an observation of stable-clusters.

In an early time, the author tried to find an algorithm of vision, to speedily and quickly trace the movement with this algorithm. His purpose was that trended an action and traced a predication by using software code. Due to the limited development, he mainly used two observable keys with centers and weights. He considered a stable-cluster could be defined as a coincided position of Its centre and weight. Then did a project about cords movement and computation with computer. Then considered a way of observation could be defined as a potential energy. Here the once be explained as a vector, also could mean as a trending movement. He continued to find a computing way of how to make an exploration with these above.

The author added a Stability, Vector and Pressure, he considered a central point of each cluster could be defined as a Stable-section. And a weight point of each cluster could be defined as a Stress-section. Then did vectorial connections where from each Stable-section to Stress-section, The author considered these vectors could be defined as PCA densities.

The length of each vectorial link could be defined as a 1 energy-status, 2 trending directions, 3 and ranged densities. Here the ranged of 2 and 3, could also determinate the statement of 1. The author continued to consider that gained a differential fissile-cluster, then could create a condition with statistics, means to do a more scaled analyzing of actions, densities, molecules and mechanics.

Finally, he thought this algorithm now only contains number of cords, weights and centers, he considered It could add one more set about area. For example, side end of cords. It might be used widely around the real world.

The author: Yaoguang.Luo 稍后优化语法.

3 轨迹状态体现在坐标团的内部欧基里德距离熵增和团中心KNN迁移熵增分析.

refer page 569, 570

4 数据预测引擎的状态机应用在非线性坐标计算系统中.

refer page

轨迹算法做漂亮些. 这个算法基础思想来自欧基里德mean距离, 讲课教授是M. klassen, Refer下, 当时讲pearson pangningtan教材章节, 讲的很仔细. 这里略。

Gitee 20200307 感想 《2维 3维 坐标集 切裂 重心 轨迹 跟踪算法JAVA源码》
1 最近我总是被一些计算思想困扰我很久, 比如数据团的切裂和融聚的模拟过程如果没有按预期的算法执行怎么办?
比如坐标团的运动趋势, 因为一些误差导致最终结果变形, 站在运维的角度我怎么及时可控, 可观测?

2 2012年上卡教授的数据分析课, 坐在赵川童鞋旁边, 想起那一副老实巴交的样子, 我问题一下子多到爆炸. 特别是rosemead去thousand oaks学校的商旅路径计算, 先走10 101高速还是先走105405~101高速.. ? 要满足我不堵车, 既看海, 又跨山需求. 2个小时到学校不迟到, 重点是60美金汽油能开1个星期. knn的增量我一定要进行轨迹可控, 不然100万次计算如果是最后10万次出错, 我只要按轨迹把后面10万 次修正, 而不是再做一次100万次计算. 提高算能是我的核心思想.

动机, 一大堆数学问题堆在一起, 我头皮发麻, 我急切需要一种算法, 来保证(计算可控, 趋势可控, 观测透明)
我定义为算能挖掘体系. 于是开始设计这类算法. 论证, 这个算法最终完善了当前版本, 能将KNN的划分重心轨迹按时间顺序记录了, 这个轨迹不但可以计算加速度, 还可以提高观测和轨迹趋势预判的准确性. 结果, 这个算法特别适用于金融领域, 运动领域加速度计算, action游戏领域, 空间几何领域等 做轨迹坐标趋势和预测计算. 不足, 因为循环计算轨迹坐标的, 团的坐标数越多, 计算量增大, 这个做算法之前先执行PCA重心团筛选函数可以大幅减少循环计算量, 提高算能.

商旅计算 离散模型预测

1 数据预测引擎的离散模型预测, 作者主要用在商旅计算中.

refer page --商旅计算在真实生产场景中用途广泛，经常用于欧拉神经网络环路的测试，基于这类预测可以提高劳动密度和计算效率。

2 作者主要用在商旅计算中的 小坐标分子群计算中.

refer page 568 --自然界常见的小分子群拟态有 蚁群，蜂群，鸟群，车流，人流，雷达，微生物团等，

3 作者的商旅计算最大价值主要体现在 欧拉环路的分析中.

refer page 568 --环路计算可以分析最短路径，而作者的动机是减少算能消耗，所以一直在这个方向上思考。

4 作者的 欧拉环路为破解 十六进制 十六元基进制编码 起到了基础研究作用.

refer page 下册56, 下册125

质量分析 概率机--稍后

1 数据预测引擎的概率机比较简单, 仅仅贝叶斯系统.

refer page --作者的量化计算和时函数的微分逻辑来自于贝叶斯优化过程得到的灵感。在每次面对各种精确的笛卡尔条件数据进行关系计算的时候，罗瑶光先生偶然发现将微分代数函数进行比率求极限的时候分解为deta N +deta C的模型，可以将快速过滤的误差堆积在deta C 中，于是得到 （2cosAcosB + -1）*detaN + detaC的噪音分析公式。这个模型公式可以应用在各种观测计算中和比率计算中。如第一章DNN部分，当前的罗瑶光DNN快速计算函数便用到了量化比值优化逻辑。减少计算算能消耗。

2 贝叶斯系统在作者的工程中很少用到, 如线性回归, 衰变失效就不包括.

refer page

3 贝叶斯系统作者有设计交叉概率机,

关于数据挖掘pangningtan教材的质量分析，讲课教授 卡拉森. refer page 616

4 作者设计概率机, 主要是之后做图片识别预测用.

refer page

坐标数据 向量机

1 数据预测引擎的向量机作者主要设计了团中心和重心的距离向量.

refer page 595 --常见的心距有 内切圆心，外切圆心，角平分，角垂直等，各有用途。

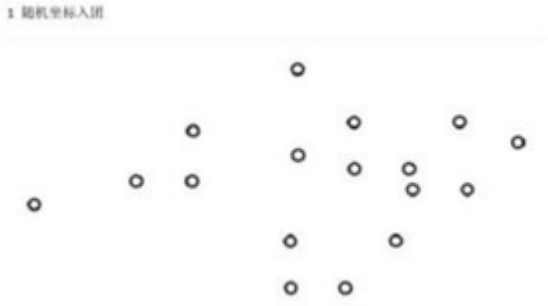
2 距离向量 可以作为路径猜测, 运动趋势, 和轨迹判断用途.

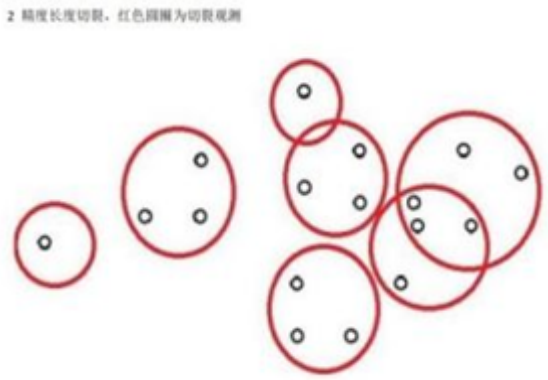
refer page 621, 624, 634

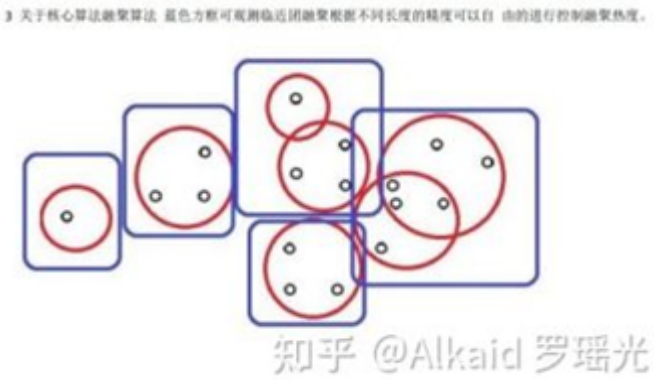
3 距离向量理解为斥力, 可以表达坐标团的稳定性评估. 可以应用于分子间的力学表达。

refer page 601

4 距离向量理解为压力, 与雷达机结合, 可以计算表达坐标团的密度. --密度大的点集蕴藏势能，可以通过时函数跟进计算。refer page 610, 613, 605, 593







德塔坐标团的密度 一般指, 将坐标进行 观测距离的区间进行划分后的坐标融聚小团, 的坐标数和团数的比值举例 如果划分有5个区间, 每个区间坐标数是 1, 3, 4, 3, 6,, 那么比值是1/5, 3/5, 4/5, 3/5, 6/5 这里的观测距离是可以精度调节的. 通过排序可以迅速计算 用于确定压力的位置. 定义归纳人 罗瑶光, 稍后优化.

欧拉环路 商旅TSP

1 数据预测引擎的商旅TSP, 主要计算随机坐标集的欧拉环路.

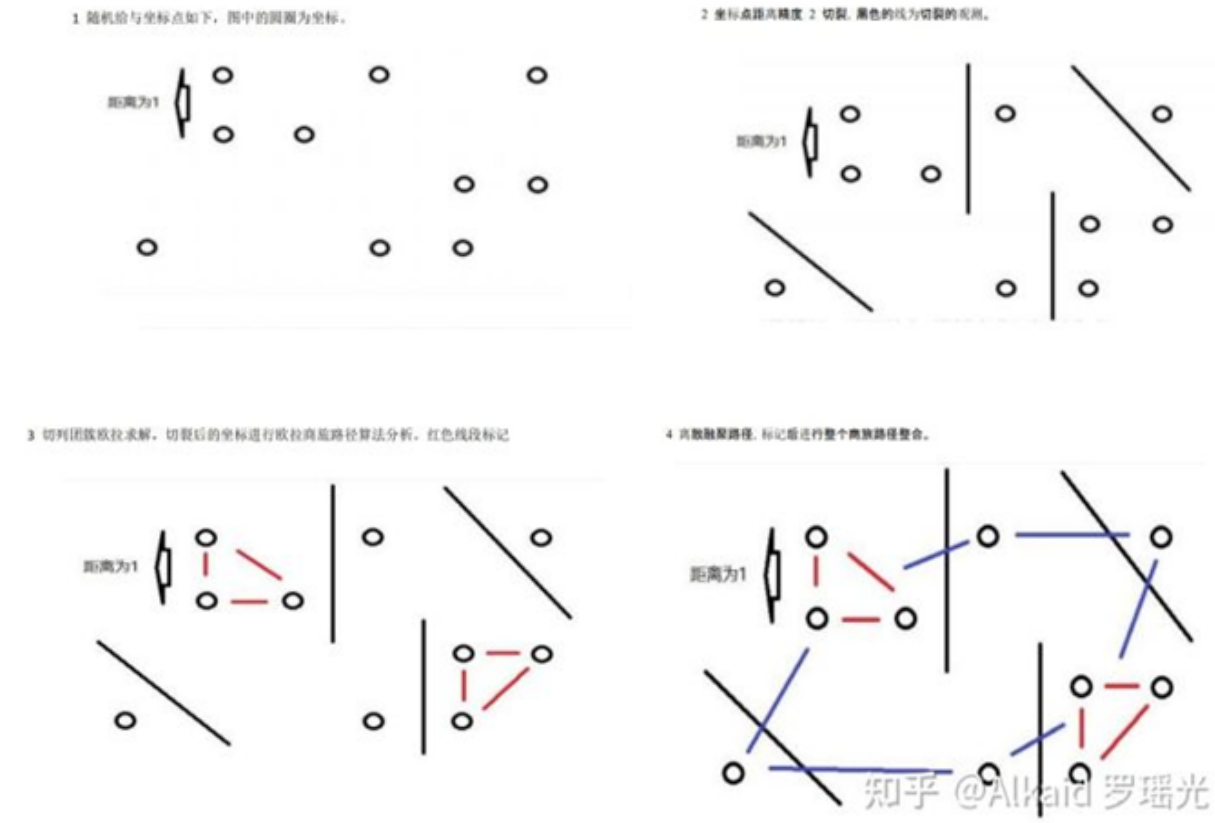
refer page 625.

2 数据预测引擎的商旅TSP, 作者设计动机为极速小分子团间的欧拉2阶图研究.

refer page 630.

3 作者研究动机为破解元基罗盘的 离散活性邻接矩阵变换.

refer page 下册5,



极速高阶欧拉融聚商旅团TSP路径算法的思维来源

作者最早接触轨迹路径算法, 来自高中玩帝国时代, 红色警戒和星际争霸游戏, 里面鼠标右键点击自动生成行军路径很酷炫, 作者的完整欧拉理论基础来自印度基督大学维嘉神父教授的离散数学第六版紫皮书. (当时胡春牛带了本中文的离散数学简化版书来了印度, 记得有欧拉图论的中文, 方便了作者的阅读). 作者第一次听到商旅算法 这个词汇来自亚米Little的口中, 当时一行人出去寻觅圣盖博下馆子, Little说研究商旅算法的都很牛. 而作者第一次看到问题, 是作者在走四方一天晚上加班翻译php那个快黄了的项目的时候, 国内来了个梁总, 莫名在作者旁边起高腔和余总争论一些琐事, 突然把我叫过去说旅行算法计算太慢, 问我怎么解决, 我当时比较懵, 余总说这是国内团队设计的, 不是作者的事. 后作者面试和求职英特尔前, 打发时间, 开始刷leecode, coderank等, 就把23~49个坐标的商旅TSP给 计算了. 后来就逐步优化. 当作者的切裂算法问世了时候, 突然想到, 如果将切裂开的坐标进行密集度融聚, 这时候融聚的坐标就是微分小欧拉, 上层就是大欧拉, 于是这个模型图便诞生了. 作者分享一个今天才揭开的秘密给大家: 切裂的词汇来自作者2012年玩苹果ios游戏 索罗门法师(Solomon Boneyard), 里面的跟踪魔法弹叫 fissile 哈哈. 罗瑶光

A Time Locus of Speed Fissile TSP about Ouler Forest

The author did a sixth foundation of Discrete Mathematics in a college time by following Father of Christ, Dr. Joseph Varghese. Before Mr. Yaoguang went to Folsom Intel, he had finished a simple Ouler TSP Algorithm to deal with the coordinates which amount less than 23 coords, then made an optimization which promoted to 49 coords. Once the DETA Fissile Algorithm had built, the author thought that made a separation of those coordinates by a long distance firstly, then did a DETA Fusional Algorithm by a short distance, and then made an Ouler TSP connection based on short distance, and finally made an Ouler TSP connection based on long distance. It seems like a Ouler Forest, and the inner short TSP distance groups such as tree clusters.

Author YaoguangLuo 稍后优化

4 作者研究结果为十六元基进制 破解 DCPE-THOS-MAXF-VIUQ.

refer page 下册5, 下册56, 下册125

十六元基进制 破解 DCPE-THOS-MAXF-VIUQ 文字描述

作者在这里主要感谢印度基督大学的财务官, 离散数学老师, 维嘉神父, 关于作者的图论和欧拉思维, 以及作者的汉密尔顿有向图的rotation变换和 adjacent map 邻接图离散变换思维全部来自维嘉神父基于离散数学第六版紫皮书的知识点传授. 作者至今保留了所有印度基督大学考试笔记卷子在浏阳保险柜中. 具体例子如下图的'元基肽展公式关系图'与 '元基语义肽展活性罗盘'的邻接变换. 描述人 罗瑶光

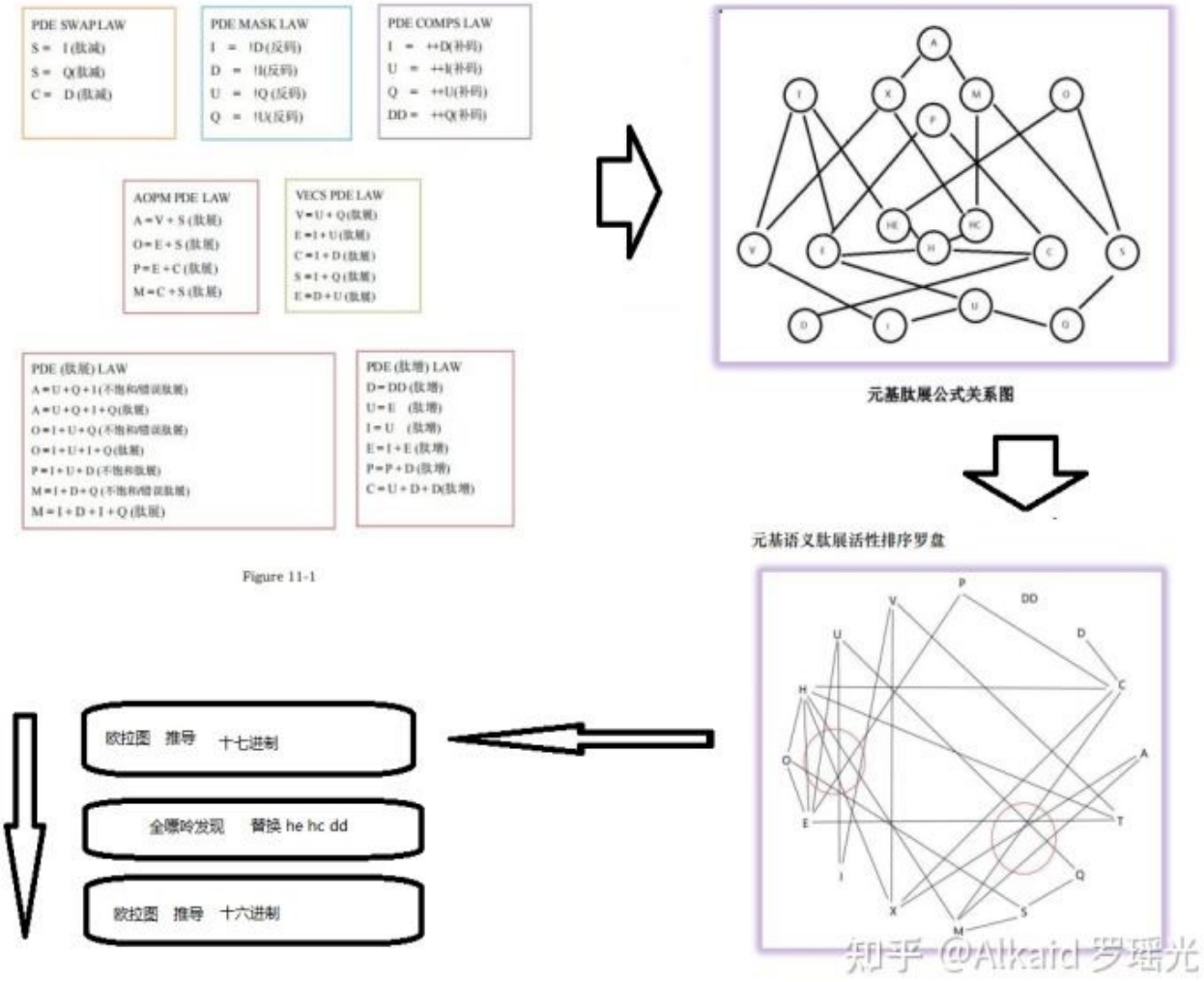


Figure 11-1

破解方式, 当作者的欧拉图算法成型后(《数据预测引擎系统 V1. 0. 0》), 首先通过DNA元基编码(《类人DNA与 神经元基于催化算子映射编码方式 V_1. 2. 2》)进行推导出语义肽展公式(《肽展公式推导与元基编码进化计算以及它的应用发现》), 然后进行按公式归纳关联方式得到十七元(《DNA催化与肽展计算和AOPM-TXH-VECS-IDUQ元基解码013026中文版本》)肽展公式关系图(《DNA元基催化与肽计算第二卷养疗经应用研究20210305》), 通过元基语义肽展活性排序罗盘观测, 开始寻找一条十七元基的欧拉路径(《DNA元基催化与肽计算第二卷养疗经应用研究20210305》), 随着全嘌呤F元基(《DNA 元基催化与肽计算 第三修订版V039010912》)的定义, 于是替换掉文中的HE, HC, DD 元基, 重新寻找一条 十六元基的 欧拉路径. 于是发现了DCPE-THOS-MAXF-VIUQ进制(《DNA 元基催化与肽计算 第四修订版V00919》).

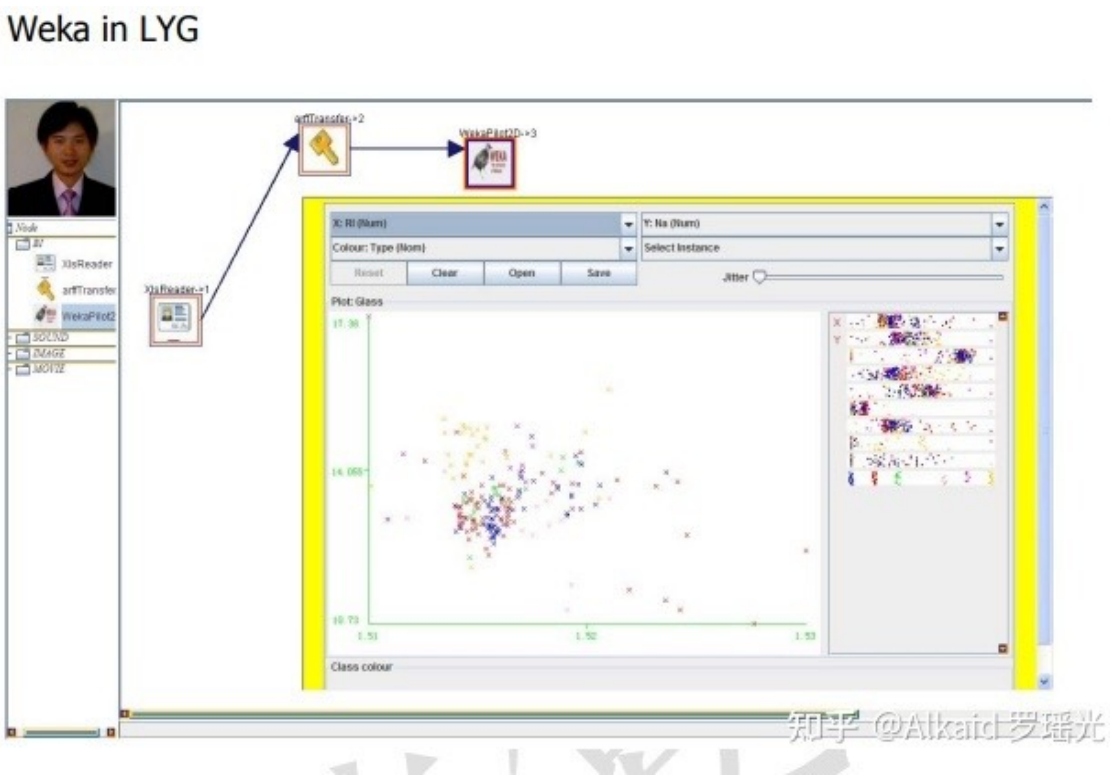
The Derivation of Hexadecimal Meta-Base, INITON. DCPE-THOS-MAXF-VIUQ.

After the Derivation of DETA Ouler TSP,《数据预测引擎系统 V1. 0. 0》. The author according to the DNA Encoding,《类人DNA与 神经元基于催化算子映射编码方式 V_1. 2. 2》. Then finds a PDN Extension, PDE formula,《肽展公式推导与元基编码进化计算以及它的应用发现》. Then continuing does the DNA Decoding to find seventeen Meta-Bases, INITON. AOPM-TXH-VECS-IDUQ-HE-HC,《DNA催化与肽展计算和AOPM-TXH-VECS-IDUQ元基解码013026中文版本》. Then try to build PDE map relationships. Base of the compass of arrangements, the author finds a septendecimal

Meta-Bases, INITON. DCPE-H(HC)XA-MSO(HE)T-VIUQ. 《DNA元基催化与肽计算第二卷养疗经应用研究20210305》. After finding a definition of Meta-Base, INITON of F, 《DNA 元基催化与肽计算 第三修订版V039010912》. The author removed the INITON of HE and HC, and by using F instead. Finally finds a new TSP link to Hexadecimal Meta-Base, INITON. DCPE-THOS-MAXF-VIUQ. 《DNA 元基催化与肽计算 第四修订版V00919》. The author YaoguangLuo 稍后优化语法.

应用

早期应用实例, 不仅在德塔自己的坐标插件可以灵活应用, detaETL 也可以集成 awt+ weka第三方插件研发 进行数据显示实现. 如下图的pilot例子. 作者早期用swt+knime进行weka设计, 自从自己写了ETL Unicorn后, 发现SWT插件都不需要了.



图中weka技术来源描述

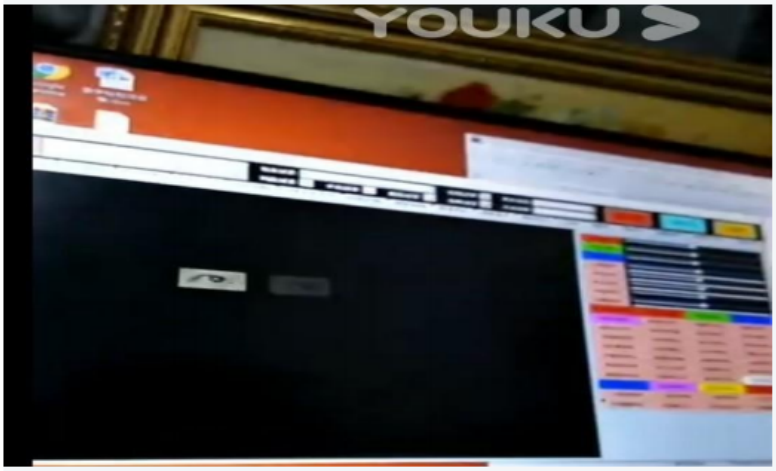
因为章鑫杰送给作者联想电脑被牛怡然同学 拿卡拉森的usb烧录操作系统给格盘了, 地点(作者和刘熠同学, 谢晨然同学, 牛怡然同学的合租房内), 西数硬盘又砸了的环境下(作者在rosemead Del Mar AVE路的租房内), 作者2013年冬在rosemead Falling Leaf设计出ETL Unicorn后, 用313699483的qq 问赵川同学 当时卡拉森的课 用的knime节点还有吗, 准备翻译成unicorn节点试试, 赵川说不见了, 于是作者当时说那就算了, 再重新写过就是了, 用poi + hssf + arff + weka api. 如上图展示, 发现效果不错. 罗瑶光

另外函数分类方法如 切裂, 融聚, 隔离, 簇类, 就不介绍了数据挖掘的聚类思想作者个人表达方法而已.



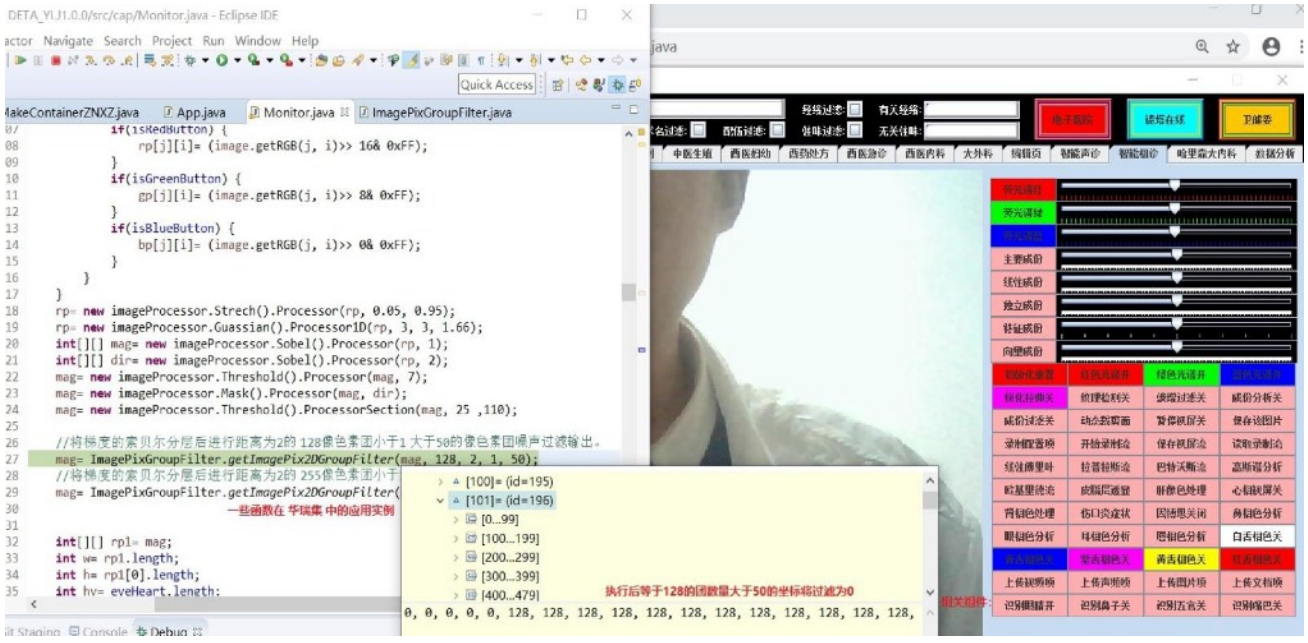
眼睛团坐标识别

https://v.youku.com/v_show/id_XNDYyMTA0Njg2NA==.html?spm=a2h0c.8166622.PhoneSokuUgc_1.dtitle





动态眼睛识别的原理



像素团处理

这个算法 动机很明显, 为了满足我的图片模式识别的工程应用.

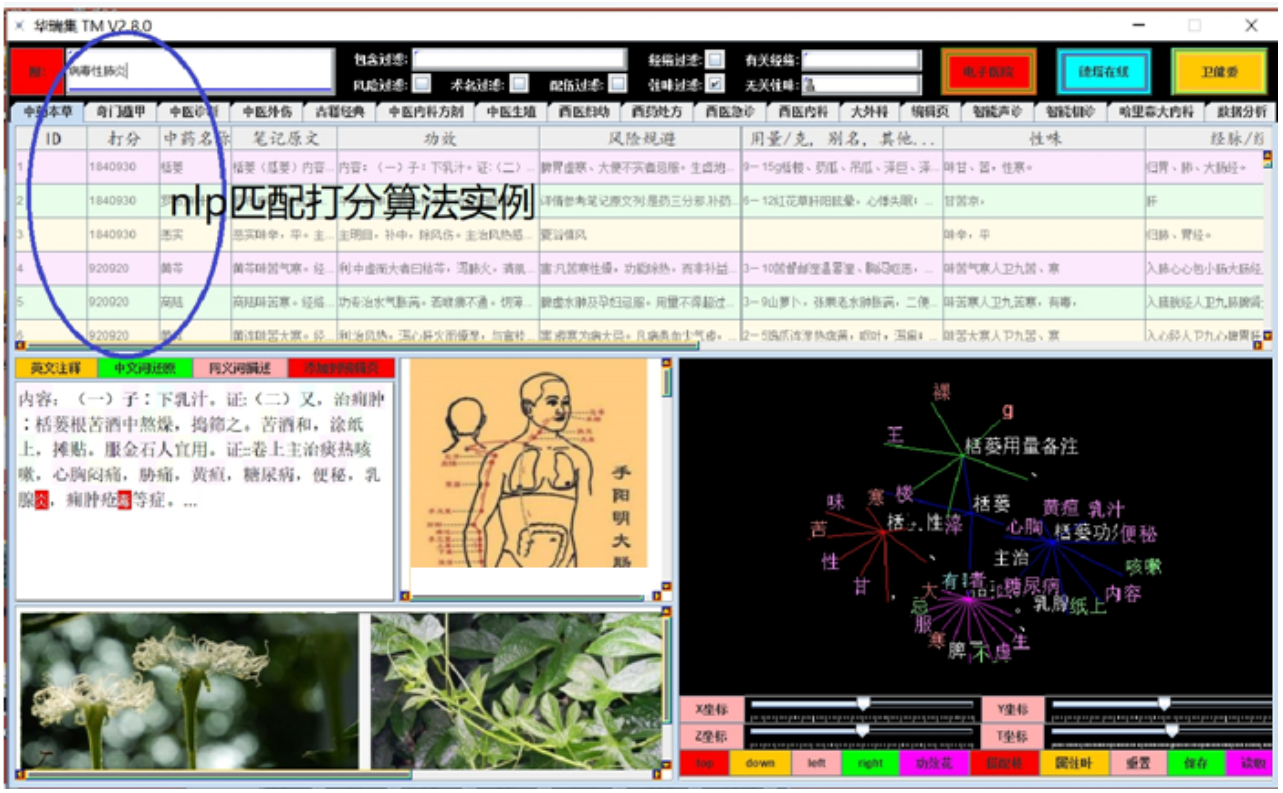
问题: 之前做眼睛识别的算法时候, 我遇到了一个问题, 就是高斯噪. 比如我先做一层粗卷积匹配, 再做精度筛选匹配(为什么要这样设计, 因为速度. 如果直接1024x 768的视频处理做50x 50的卷积, 这速度让我痛苦因为循环次数是1024x 768x 50x 50次, 我迫切需要加速, 于是先通过5x5粗卷积识别眼珠, 再10x 10细卷积识别眼白, 再25x 25.. . 识别眼眶), 发现眼睛的识别是准了, 速度快到爆炸(循环词次数是 1024x 768x 5x 5x(条件概率事件), 最大比值是2500/ 25, 100倍的差值), 但是总有那么几个噪声同样被识别出来, 我需要 把这些东西给过滤了, 迫切需要一种模式识别算法.

解决: 常规的算法和高斯模糊算法我都组合尝试遍了, 结果还是没有令我满意, 我觉得有必要基于燥的小分子团的大小 观测面上进行过滤这些噪声成份, 这是最早的动机.

论证: 算法设计完后, 的确可有效删除某特定像素的 分子团, 因为一些卷积特征相同的 区域就很好地被过滤了.

不足: 因为是基于scale距离来划分分子团的, 所有一些环形的同色分子团因为重心距仍很长, 不能有效地过滤. 之后会专门讨论这种情况. 另外计算过程观测时 发现卷积计算之前就用该算法过滤, 动态视频处理速度无影响, 如果在粗卷积匹配后再做这个算法过滤, 速度无法保障, 这是缺陷, 我会进一步完善, 或者研发更有效地分子团过滤算法.

算法搜索的NLP匹配打分



这一章，我用的来推导元基欧拉进制环路，但是发现内容功利性比较强，所以不做过多思考描述，就留1个问题

思考 本书第6章节逻辑和当前微分拓扑学的教材内容逻辑有什么出入。大学3年级难度。

思考实践下常见的软件滤噪技术方法和种类，大学3年级难度。

第七章 类人DNA与神经元基于催化算子映射编码方式

The INITON Catalytic Reflection Between Humanoid DNA and Nero Cell

类人 DNA 与 神经元基于催化算子映射编码方式

Yaoguang Luo, Rongwu Luo

罗瑶光, 罗荣武

Keywords

VPCS, AOPM, IDUC, Nero, Artificial, Decoder, Medical, Paralling, Computing,

Humanoid, ETL, Parser, Data Mining

关键词

VPCS 架构 AOPM 逻辑 IDUC 编码 神经元 人工 解码 医学 并行计算 类人仿生 ETL 数据挖掘 爬取

Outlook: VPCS architecture is not the end. Absolutely, at least at this paper, I will make an implementation in five sections: DETA humanoid cognition, DETA Medical Business backend logic, DETA Catalytic computing, DETA Finding initions, DETA DNA decoding. Above all I also will spend more and more words in my DETA DNA Law of IDUC. And its applications in the real world. Ok next step as below.

观点: DNA 的本质是一种智慧体对数据增删改查的四个元操作的组合方式编码索引链. VPCS 架构不是德塔最终架构, 观点很明确, 至少这篇文章中, 作者会做一个详细的论点论证, 基于5 个部分, 德塔工作在类人认知方式, 医学商业逻辑的后端处理, 催化计算的过程, 德塔计算算力本质的寻找, 以及仿生DNA 的解码分析, 最后, 作者会组织很多的文字来描述德塔的一些振奋人心的发现: 比如 DNA 的编码元基如IDUC 对应增删改查, 以及它的真实环境的社会应用实例.

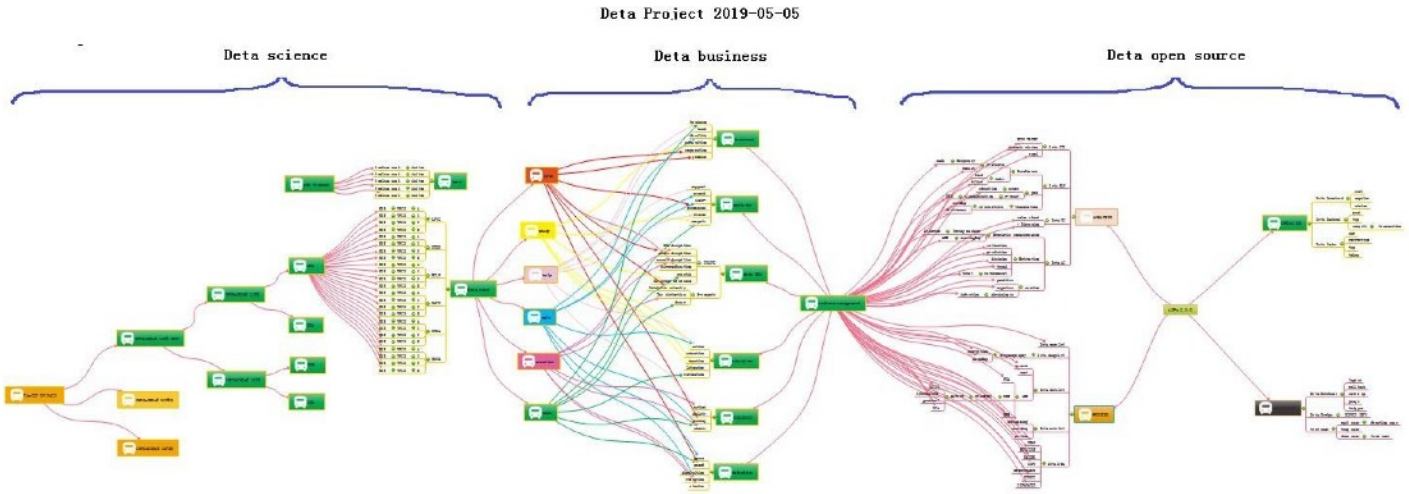
1 DETA humanoid cognition

Since the inition of the DETA OSS, there has a lot of questions were based on the humanoid DNA catalytic computing, I have been working on this domain for a long time. Absolutely also, I have got a lot of flashing points here, for example AI, still remember the first time I touch the cognition this verbal at CLU Dr Reinhat's class about cogs PU computing and cognitive quality Sonar test where in Folsom Intel, I know that cog-work is a trending task in my life.

德塔开源的起源, 建立在一堆疑问之上, 作者有必要进行解释, 人造人? 永生? 自我进化? 答案有很多, 无疑都是正确的答 案, 德塔一直努力在创造一个基础数据计算体系, 让技术更好地适应生活环境. 通过多年的努力, 德塔找到了一些振奋人心的答 案, 比如 对环境的认知能力, 这种能力用计算机语言来描述就是 基础算法, 人工智能的启蒙. 作者第一次 接触认知 这个词汇, 是在加州路德大学Reinhart教授的cogs多核芯片编程的课和 英特尔 福森总厂 工作的时候 接触了 Sonar Lint 的 认知测试, 当 时感觉 认知是一个趋势. 虽然 sonar 的作用是让代码更简洁有效, 保证代码书写的规范性. 作者感觉认知方式 的 基础灵感 很多 应该来自于源码的规范.

Eventhough, I need do a lot of basic foundations at AI, but I insist to know, from the code where between normalization and duplication. I have been thinking how to make a contraction and distinction AI logic with human and humanoid. Ok, plan a start as below. Code a problem's solution of software like a way of Yaoguang Luo 's cog-style life.

通过 sonar 给作者的灵感, 作者一直在思考, 如果 sonar 中的规范成分能塑造人的行为规范, 那人 劳动方式和代码的执行 方式是一样能够准确定义的. 于是一个计划在作者的心里萌芽. 编写人类劳动的认知规范. 怎么写? 作者有一个明确的思路, 用自 身对世界的认知来重塑一个规范. 逐步优化它. 于是 源计划 开始.



Look at this PIC, smartly, build basic foundations first, then create more business software were based on these foundations, and finally swap to a humanoid mode. Many many times I hope the mode could be an Immortality.

如图的的工作非常的明确, 先设计开源基础, 然后进行商业论证, 最后催化编码设计类人进化智能. 截至到公元 2020 年 10 月 已经实现了 18个德塔数据智能子工程 开源, 6 个独立的知识产权软著, 2 个医学大数据辅助学习诊疗系统, AOPM VPCS 的 INITON 催化编码规范 7 篇论文. 进度有条不紊. 感谢一切. 很多时候我所作的一切 我希望是别人所期盼的一切. 正如永生算法.

1. 1 DETA humanoid cognition history, 德塔类人认知历史

In the past, knowledge of the world could be parsed by five sections, the world's cognitive way, philosophers and scientists liked to describe objects with five senses (touch, taste, hearing, smell and vision). Species sense could make creatures in order to adapt to the environment well, understood the environment, and thought about ways to protect themselves in dangerous environments. These methods were implemented in a variety of ways, and their execution logic was also varied, but the causes and results were clear about a basic point. Better adapt to the environment. During the process of designing the YangLiaoJing, the author had well integrated the bionic technologies of voice, text, association and visual media, and had been optimizing them to gradually form a comprehensive intelligent YangLiaoJing system.

过去, 世界的认知方式 哲学家和科学家 喜欢用五感(触觉, 味觉, 听觉, 嗅觉, 视觉)来描述物体的方式, 物种感觉能让生物很好地适应环境, 理解环境, 能够在危险的环境中思考保护自身的方法. 这些方法的实现方式丰富多彩, 执行逻辑也各种各样, 可是起因和结果都明确了一个基本点, 更好地适应环境. 作者在设计养疗经的过程中, 很好地融入了 语音, 文字, 联想, 视觉媒体的仿生技术. 一直在优化它们逐渐形成一个综合智能养疗体系.

1. 2 DETA humanoid cognition development, 德塔类人认知研发

In order to better adapt to the environment, human beings began to create characters, invent tools and improve their cognitive ability, from the ignorance of slaves to the open and compatible world, from the Iron Age to the current nano-chip technology. In the river history of 5000 years, human beings seem to have evolved aimlessly. Through these phenomena, the essence could be easily discovered. Better adapt to and transform the environment. The best arguments to improve the cognitive ability of environmental things are the research and development of basic science and technology and systematic induction, Marconi's wireless telegraph, Zu Chongzhi's pi, Darwin's origin of species, code of Hammurabi, and countless outstanding scientists, thinkers, inventors and philosophers in history. These people seem to be shining stars in the night sky. Gradually, Intelligent creatures begin to have enough ability to look up into space and explore the mysteries of the universe, and this enough ability is the basic ability to improve the cognition of things, which is very important. The Huaruiji system of DETA Company draws lessons from the systematic induction method of human instinct and dialectically treats medical diseases with medical textbooks as the cognitive basis, which is in line with the embodiment of scientific development.

为了更好地适应环境, 人类开始创造文字, 发明工具, 提高对事物的认知能力, 从奴隶的愚昧制度到 现在开放兼容的世界, 从铁器时代 到现在的纳米芯片技术. 人类 在 5000 年的历史中似乎是漫无目的的思维进化, 透过这些现象, 本质其实很容易被挖掘到, 更好地适应环境和改造环境. 最有效地提高对环境事物的认知能力, 最好的论据是 基础科技的研发和系统性归纳, 马可尼的无线电报, 祖冲之的圆周率, 达尔文的物种起源, 汉谟拉比法典, 历史上出现无数卓学的科学家, 思想家, 发明家, 哲学家, 这些人似乎是夜空中璀璨的明星, 逐渐, 智能生物开始有足够的能力仰望太空. 探索宇宙的奥秘. 而这个足够的能力便是提高对事物认知的基础能力, 至关重要. 德塔公司 的华瑞集系统 借鉴人类本能的系统性归纳方法, 将医学教材 作为认知基础来对医学疾病辩证, 是符合科学发展的体现.

1. 3 DETA humanoid cognition application, 德塔类人认知应用

There are many applications of humanoid cognition in social science, and there are many excellent arguments here, such as auxiliary auditory system, big data reasoning system, weather forecasting system and criminal investigation database system, which undoubtedly proves that cognitive mode has greatly improved the adaptability of human beings to the environment. To the ability to gradually transform the environment from part to whole. There are many outstanding arguments here, from the ancient Dayu's flood control, to the renovation of Sui-Emperor Yang's Canal, (当前政治经济实体实例已经过滤), from artificial rainfall to artificial islands. The argument is very clear, and the cognitive ability of things comes from the accumulation of basic science and technology. These basic technologies gradually form a system. On top of It are a wide range of scientific and technological commodity applications, (当前政治经济实体实例已经过滤), giant hydropower station of Three-Gorges of Yangtze River to improve geographical environment, etc. There are too many. The evolution of human wisdom gradually forms a clear route, and improving basic science and technology and cognitive ability complement each other.

These abilities all come from thinking about things. Then form a solution, and finally implement it. This process is summarized as the process of analysis, operation, processing and management. The life cycle of software engineering is well explained here. Analysis A, Operation O, Processing P, Management M, use simple words to describe that even if unknown data are collected and analyzed, and then things are operated, the solutions to various difficulties encountered in the operation process are implemented. Finally, maintain and manage these implementation experiences. The back-end computing mode and system life cycle of DETA have gradually condensed from the earliest collection, analysis, operation, sorting, coding, running, debugging and maintenance to the module modes of Analysis A, Operation O, Processing P and

Management M, such as DETA word segmentation, DETA DNN mind reading, etc. Now ETL of DETA is ready to go in this direction. The author designed a paper last year to describe the application mechanism of AOPM as follows:

类人认知在社会科学上的应用很多, 这里有很多优秀的论据, 比如辅助听觉系统, 大数据推理系统, 天气预报预测系统, 刑侦数据库系统, 太多了, 这无疑论证了认知模式大大提高了人类对环境的适应能力. 更确切的语言表达是 从提高适应环境能力, 到逐渐进行局部到整体改造环境的能力. 这里又有很多卓越的论据, 从上古时代的大禹治水 到 隋炀帝运河改造 (当前政治经济实体实例已经过滤). 从人工降雨 到 人工岛屿, 论点非常明确, 对事物的认知能力来自基础科技的积累. 这些基础科技逐渐形成一个系统, 在它之上是广泛的科技商品应用, (当前政治经济实体实例已经过滤). 等等, 太多了. 人类的智慧进化逐渐形成一条明确的路线, 提高基础科技与认知能力相辅相成. 这些能力 都来自对事物的问题 产生 思考, 然后形成 解决方案, 最后落实, 这个过程作者归纳为分析, 操作, 处理, 管理的 过程. 软件工程的生命周期在这里有很好的解释. 分析A, 操作O, 处理P, 管理M, 用简单的话语来描述即使将未知 数据进行采集分析, 然后进行事物化操作, 操作过程中遇到的各种困难的解决方案落实, 最后维护和管理这些落实 经验. 德塔的后端计算模式和系统的生命周期从最早的 采集 分析 操作 整理 编码 运行 调试 维护 逐渐浓缩 成 分析 A, 操作 O, 处理 P, 管理 M 的 模块模式, 比如德塔分词, 德塔DNN 读心术等, 现在德塔的 ETL 也准备往这个方向走. 作者去年设计了一篇论文很好地描述了 AOPM 的应用机理如下:

AOPM Open-Source System on SDLC Theory

Mr. Yaoguang.Luo

Outline: Mr. Xuesen. Qian once said: 'Science Is a Titan System', as an open-source software conception. This topic implements a software interaction theory of SDLC for Analysis, Operation, Process and Management— AOPM. Also, this is a tiny paper where easy to show more idyllic landscapes of using DETA open-source projects. Not only for web system, also for mobile and desktop platform. The final goal makes complex project to simple. Ok let go and the next steps.

观: 钱学森先生曾经说过科学是一个巨系统. 结合开源软件的设计理念, 这篇论文提炼出软件生命周期的核心部分, 如采集分析, 细化操作, 执行处理和运维管理的四个部分, AOPM, 这篇小论文不仅适用于开源工程, 在互联网或者桌面系统上, 甚至适用于各种复杂的巨系统.

Keywords: SDLC, AOPM, VPCS, WEB, Concurrent, Open Source, Interaction, Management, Automation关键词:软件生命周期, AOPM, VPCS, 互联网, 并发, 开源, 交互, 管理, 自动化

Introductions. Recently my colleagues take more care on the SDLC evolution of open-source software engineering, for each project they undertake on where It cost a lot of times, that's for my job, continuing found out a high effect, simple and clear theory of SDLC what be my main task now. after imagination and logic recursion, the key is an optimization of ordinary SDLC such as water fall. First time for makes an introduction to waterfall of SDLC? The author's explanation likes sequence linked list of component nodes. With DETA projects here contains four aspects at Figure1-1. And my explanation of open source as belows.

介绍: 最近, 我的朋友总是关心软件生命周期的演化, 这些琐碎的工程事物, 不仅浪费时间, 也降低效率. 持续地寻找一种高效的清晰的生命周期, 纳入了我的个人日常工作范围. 通过大量的细化和 传统的基础理念优化, 我觉得软件生命周期的瀑布模式是一个很好的参照点, 可以进行发掘归纳如下.

Topic: Ten Definition of The Open Source, OSS Book Reading Note/这是我在路德大学研 3 写的读后感作业

In this paper, through a premise: the contrast between the copyright and the contract. The author talks a comprehensive introduction to the definition of the open-source code. The role of the open-source licenses, which is allowing the work permit under the non-exclusive business. Not only does It mean that the source code was visited by the public user, and also meets another 10 conditions as follows. The first point: the open-source software allows the free reusable distribution. The license must not restrict that any party sell or give away the software. At the same time, it can't get the sold fees and other fees for this software. The second point: the program must include the full of source code. The license does not allow that getting the source code from any specific forms of the production. The license assures that no one can intentionally to confuse the source code. At the same time, users have rights to access to the source code under this license. The third point: which talks about the rights of the derivative work. The license must allow the work-modification and the new-work-derivation. Those new's are published under the same license. The fourth point: the integrity of the source code. Licenses and the integrity of permits, which may limit the distribution of the form of the modified source code. The fifth point: license does not discriminate against any specific groups and individuals. The sixth point: license does not limit the use way of any particular field scheme. At the same time, the license can't limit the use way's flexibility and reliability. The seventh point: the distribution of the license. Distribution solutions do not need additional license. The eighth point: the license must not specific to the product. The redistribution of the software does not dependent on the program. The ninth point: license may not restrict other software. This license may not restrict the publish of the software. The distribute software will be built by using open source. The end point: license rights is neutral. So, it effective limits that the freedom of the code transmission. In other words, it provides the preventive measures.

本文通过一个前提:著作权与合同的对比. 作者对开放源代码的定义进行了全面的介绍. 开放源码许可的角色, 即允许工作许可下的非排他性业务. 它不仅意味着源代码被公众用户访问过, 而且还满足以下 10 个条件源自于 开源协议的摘录. 第一点:开源软件允许免费的可重用发布. 许可证不能限制任何一方出售或赠送软件. 同时, 无法获得该软件的销售费和其他费用. 第二点:程序必须包含完整的源代码. 许可证不允许从任何特定形式的产品中获取源代码. 许可证保证没有人可以故意混淆源代码. 同时, 用户有权访问本授权下的源代码. 第三点:关于衍生作品的权利. 许可证必须允许作品修改和新作品派生. 这些新版本都是在相同的许可证下发布的. 第四点:源代码的完整性. 许可证和许可证的完整性, 这可能会限制发行形式的修改源代码. 第五点:许可证不歧视任何特定的团体和个人. 第六点:许可证并不限制任何特定字段方案的使用方式. 同时, 许可证不能限制使用方式的灵活性和可靠性. 第七点:许可证的发放. 分发解决方案不需要额外的许可证. 第八点:许可证不能特定于产品. 软件的重新发布并不依赖于程序. 第九点:许可不能限制其他软件. 本授权不能限制软件的发布. 分发软件将使用开放源码来构建. 终点:许可权利是中性的. 因此, 它有效地限制了代码传输的自由. 换句话说, 它提供了预防措施.

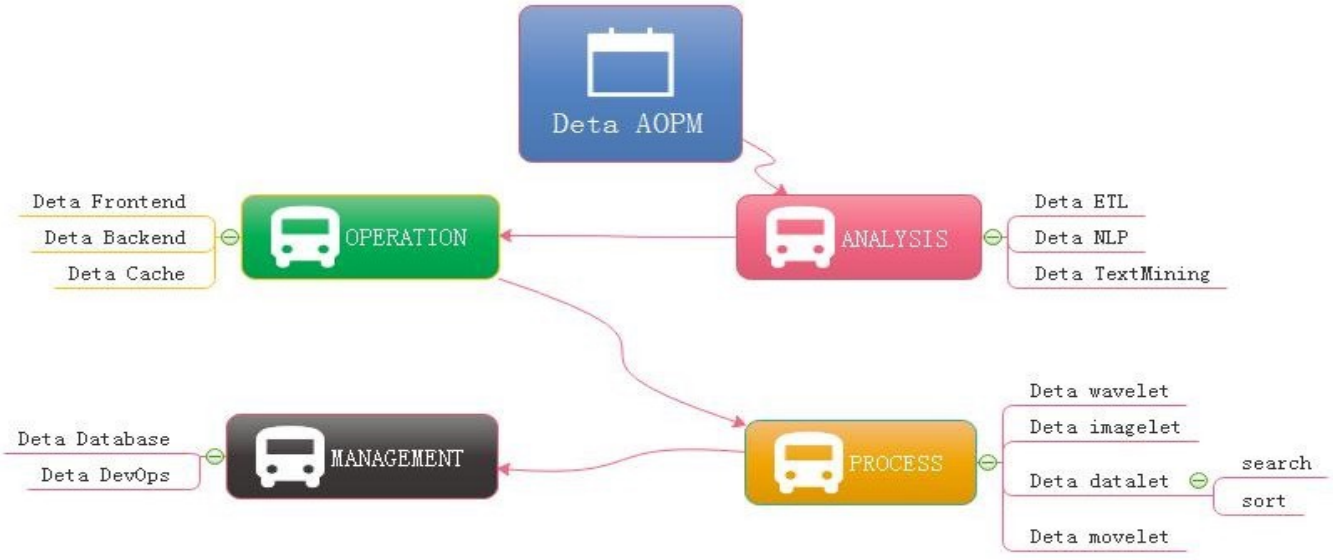


Figure 1-1 AOPM Applications with SDLC Evolutions

Last year I was asked by so many engineers but almost the same question: how have you done so many projects during the year of 2018~2019 My answer is absolutely: connection. Always, with connection, I got lots of fantasy inspirations on the projects where I undertook. My projects all are lower basic technical factors, with connections, what support me the necessary energy for continuing development on my projects. What means connection? Be an internal union bridge between my projects. For example, DETA NLP and DETA ETL, they both have the same attributes such as AI, Analysis and Data etc, with these connections, my tasks became more dynamically. Every time before I made a decision of priority levels of my projects, I thought the connection first, DETA projects totally can be separated into three dimensions. Frontend Backend and Storage, as the Figure 1-2, the connection between DETA projects is WEB AI, now is a Bazaar requirement, but we will easy to make estimation of Its future, toward to Cathedral.

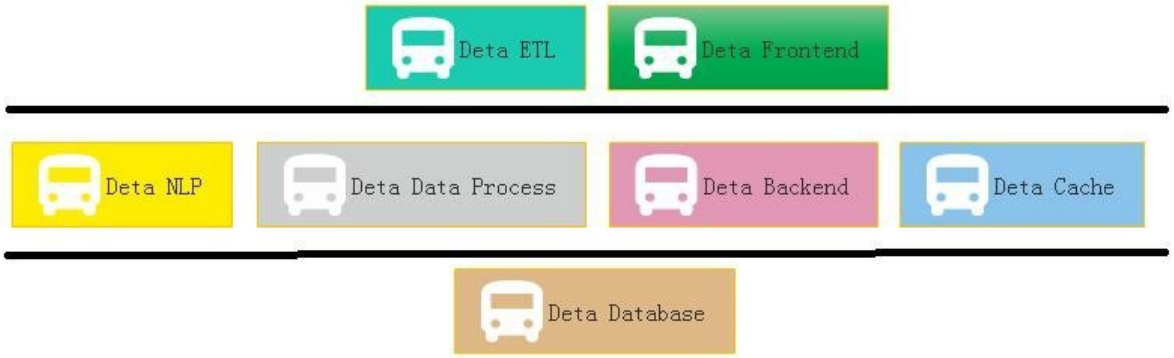
去年, 很多工程师几乎都是问过我同一个问题:在 2018-19 年, 你们是怎么做这么多项目的? 我的答案是绝对的: 联系. 在我所从事的项目中, 通过 connection, 我得到了很多幻想的灵感. 我的项目都是较低的基本技术因素, 与连接, 支持我必要的精力, 继续发展我的项目. 连接是什么意思呢? 在我的项目之间做一个内部联系的桥梁. 例如DETA NLP 和 DETA ETL, 他们都有相同的属性, 如 AI, 分析和数据等, 通过这种连接, 我的任务变得更加动态. 每次在我决定项目的优先级之前, 我都先考虑连接, DETA 项目完全可以分为三个维度. 前端后端和存储, 如图 1-2 所示, DETA 项目之间的接是 WEB AI, 现在是一个市集的需求, 但是我们会很容易的估计它的未来, 走向大教堂.

Topic: Cathedral and the Bazaar, OSS Book Reading Note 这是我在路德大学研 3 写的读后感作业

Cathedral and the Bazaar, this article has a profound implication, the author is a computer scientist with extensive experience. We can say that he is one of the early code and program contributors in the UNIX system. This article describes the Linux development with the revolutionary road, as the process from the bazaar to the cathedral. First, the author tells the contrast between UNIX and Linux: now UNIX is still popular around the world. Its rigorous structure and contribution to science, let It is proud of the same dignity as a church. Linux looks like a noisy bazaar, the code work in various countries around the world, solving their own problems and arguing in the forums and communities. Like a bazaar. Then, author points an internal factor to get an in-depth discussion: UNIX reason why It has the church's authority, because Its development has always been tailor-made by the world's most senior and most eminent researchers and software scientists. Although the discussion, because of the nature of the project-oriented, so that UNIX has been applied still to today. Even of the unreasonable original design, through decades of use, engineers have become accustomed to this experience now, there fore, we are called transcendental. which makes UNIX feels like a cathedral. The birth of the Linux was different, survival in an all-spittle environment. Every update, are implemented in controversial circumstances. The crowd here, are huge number of scientists, or writers, or code workers or merchants, their common ideal is that make Linux development meets the needs of all groups. Similarly, a huge bazaar. The author commenced a lenovo, a conclusion that Linux will eventually beat UNIX, UNIX gets the range of fresh blood is less than the Linux' s, also the number of the UNIX team members is less than the Linux' s. UNIX customers and employees are aging. But Linux development more in line with the user of the needs. Its own development is to establish a relationship on this demand and requirement. Linux is young now. Summary, UNIX and Linux development option is the two kinds of very different road. These processes and methods to determine the fate of the two kinds of software development. Of more optimizing about Linux because It is better adapted to the environment.

大教堂和集市, 这篇文章有着深刻的寓意, 作者是一位经验丰富的计算机科学家. 我们可以说他是 UNIX 系统中最早的代码和程序贡献者之一. 本文描述了 Linux 开发的革命之路, 从市集到教堂的过程. 首先, 作者讲述了 UNIX 和 Linux 之间的对比:现在 UNIX 仍然在世界范围内流行. 它严谨的结构和对科学的贡献, 让它自豪地拥有与教会同样的尊严. Linux 看起来就像一个嘈杂的集市, 代码在世界各地的不同国家工作, 解决他们自己的问题, 并在论坛和社区中争论. 像一个集市. 然后, 作者指出了一个内部因素进行了深入的讨论: UNIX 为什么拥有教会的权威, 因为它的开发

一直是由世界上最资深和最杰出的研究人员和软件科学家量身定做的. 尽管在讨论中, 由于项目的性质是面向的, 所以UNIX 至今仍被应用. 即使是不合理的原始设计, 经过几十年的使用, 工程师们现在已经习惯了这种经验, 因此, 我们被称为超越. 这让 UNIX 感觉就像一座大教堂. Linux 的诞生是不同的, 生存在一片唾沫横流的环境中. 每次更新, 都是在有争议的情况下执行的. 这里的人群, 是大量的科学家, 或作家, 或代码工作者或商人, 他们的共同理想是使 Linux 开发满足所有群体的需要. 类似一个巨大的集市. 作者开始了雷诺-沃, 结论是 Linux 最终会打败 UNIX, UNIX 得到的新鲜血液范围小于 Linux 的, 而且 UNIX 团队的成员数量也小于 Linux 的, UNIX 的客户和员工都在老化. 但是 Linux 的开发更符合用户的需求. 它自身的发展就是在这一需求和要求上建立起一种关系. Linux 现在还很年轻. 总结, UNIX 和 Linux 的开发选择是两种截然不同的道路. 这些过程和方法决定了两种软件开发的命运. 对 Linux 更加乐观, 因为它更好地适应了环境.



At figure 1-2, DETA open source main based on AI domain, It already formed as an ecology system, go ahead to the application, thanks.

Figure 1-2 Sections of DETA Projects Group Applications One question is my friend asked me why does DETA support the e-commence logic? Definitely! Please see the Figure 1-3, this is a classic horizontal deployment sample of the real word. Alibaba, Amazon, Ebay and JD etc, all based on this technology, instead of Spring, DETA can be the next generation of technology.

我的一个朋友问我 DETA 是否支持电子商务逻辑? 当然! 请参见图 1-3, 这是一个经典的服务器横向扩展部署示例. (当前政治经济实体实例已经过滤)等都是基于这种技术, DETA 可以取代 Spring 成为下一代技术.

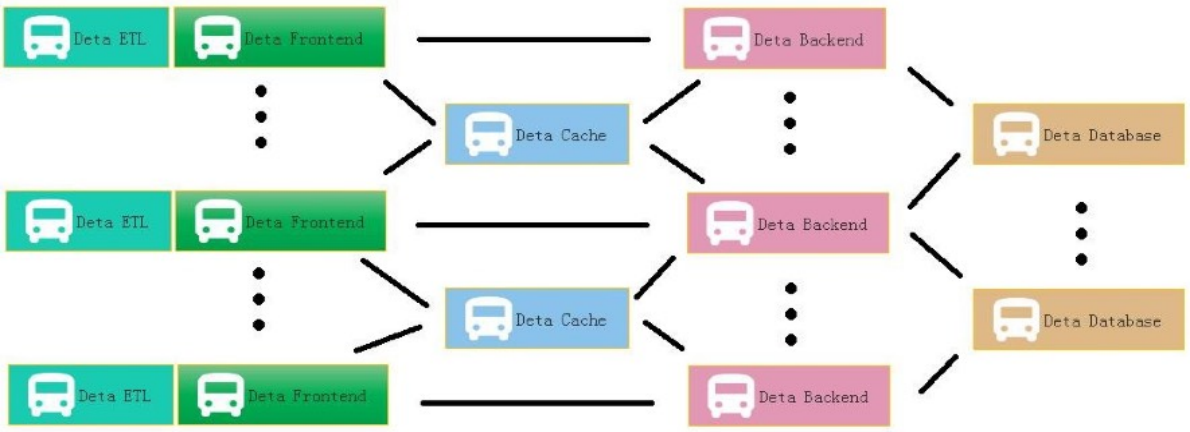


Figure 1-3 DETA WEB Projects System At Figure 1. 4 is a real sample for web Devops by using DETA Open Source.

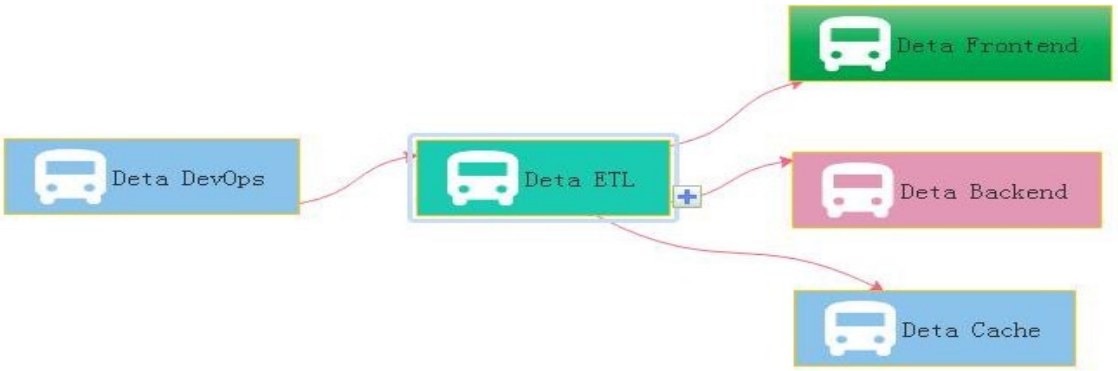


Figure 1-4 DETA DevOps Projects System

2 DETA Business back-end logic

Before 2010, the author systematically contacted the mechanism of analyzing A, operating O, processing P and managing M in the learning process. After graduation, he had the opportunity to deal with the business logic corresponding to these things, through programming in some software companies in the society. From the research of MP6 mail system (当前政治经济实体实例已经过滤), The Bluetooth group advertisement machine to (当前政治经济实体实例已经过滤), from the e-commerce back-end calculation(当前政治经济实体实例已经过滤)to the global hotel reservation (当前政治经济实体实例已经过滤), the author has been thinking about how wonderful It would be if the front-end system could give wisdom like human beings. So, the bud in the author's heart began to take root, and he was confident to design a set of architecture system with humanoid wisdom to meet the rapid development of business, intelligence and applications.

2010 年以前, 作者在学习的过程中系统地接触了 分析 A, 操作 O, 处理 P, 管理 M 的事物机制, 毕业后有机会在社会上的一些软件公司通过编程来处理这些事物对应的商业业务逻辑. 从法国(当前政治经济实体实例已经过滤) MP6 邮件系统研究, 到 (当前政治经济实体实例已经过滤) 蓝牙群发广告机, 从(当前政治经济实体实例已经过滤)电商后端计算到(当前政治经济实体实例已经过滤)全球酒店预订, 作者 一直在思考, 如果 前后端系统能像人一样赋予智慧, 多么美好. 于是作者心中的萌芽开始扎根. 有信心设计一套具备类人智慧的架构体系, 满足高速发展的商业智慧应用.

2. 1 DETA Business backend logic history, 德塔商业后端逻辑历史

The first contact with the Frontend and Backend separation was in 2004, when the author first published a website in Liuyang city by using the 7-week platform. The website was a second-level domain name, using a third-party server, even though the concepts of front-end and back-end were ignorant at that time. The author first contacted MVC architecture in Shanghai Fanteng Information Technology Company. At that time, the feeling was that MVC could solve all kinds of business logic. In the same year, the author first came into contact with MVP to do multi-thread Bluetooth big file project, and felt that MVP seemed to make the architecture handle the problem of concurrent computing well. From 2014 to 2017, the author worked almost with business logic corresponding to various MVC architectures, such as Spring, Martini, etc. The author considered that ‘gives MVC an intelligence urgently’.

第一次接触前后端分离是在 2004 年 作者第一次用 7week 平台在浏阳发布网站, 网站是一个 2 级域名, 采用第三方服务器, 第一次接触前后端的概念们当时很懵懂. 作者第一次接触MVC 架构是在上海帆腾信息技术公司, 当时的感觉是 MVC 能解决各种商业逻辑. 同年作者第一次接触 MVP 在做多线程操作蓝牙大文件项目, 感觉也很美好, 觉得MVP似乎能让架构很好地处理并发计算的问题. 2014- 2017 年, 作者的工作时间几乎和Spring, Martini, 等多种MVC 架构对应的商业逻辑打交道. 作者认为, MVC 赋予智能 迫切.

2. 2 DETA Business backend logic development, 德塔商业后端逻辑发展

Thanks to my father, in 2018, he told me to design a pharmacy-assisted search software according to the concept of Chinese medicine, so he began to design Huaruiji Medical Big Data System. At that time, I thought spring boot, mysql were too heavy. If the database rest handshake system of socket stream was designed according to CGI, many problems could be solved easily. So, I began to analyze, operate and deal with the problems. Gradually found some irreplaceable primitives, such as S static data, V visionary observation mode, P procedural registration mode, C control unit, etc. It would be wonderful if we could redesign a set of architectures for these primitives. The PC separation mode here came from an IOC doctoral design paper in Spring in 2015. Thanks here, I integrated MV into V observation mode, and then took out the corresponding static data of M and function S. This VPCS structure shoots me at present.

感谢父亲, 2018 年对我说按照中医理念设计一个药学辅助搜索软件, 于是开始设计华瑞集医学大数据系统, 当时我觉得spring boot, mysql 都太重了, 如果根据CGI 设计socket 流 数据库 rest 握手系统, 那么很多问题都好解决, 于是我开始问题分析, 操作, 处理, 找到了 一些无法替换的基元, 比如 S 静态数据, V 观测模型, P 注册方式, C 控制单元等等, 如果能将这些基于重新设计一套架构, 那该多么的美好. 这里的 PC 分离模式, 来自 2015 年 Spring 的一篇IOC 博士设计论文, 这里表示感谢, 我把MV 综合成V 观测模型, 然后把M 的对应静态数据和函数 S 拿出来, 这种 VPCS 结构目前满足了我所有的德塔医学大数据应用.

With regard to the excessive description of VPCS, I can take a previous note as follows: VPC architecture programming thought, software programming for many years, accumulated some thoughts on program realization. Through the certification of Darwin's theory of evolution, an effective VPC programming concept is elaborated based on the neutral coupling of MVC+ MVP. V is an observer mode, similar to storage object and observation mode. P is the processor, which handles the registration interface. C is the control machine, which describes and classifies the registration interface. S static control machine, why use static control machine, advantages: 1. Because of the separation of PC, the functions of C mode are inherited through abstract virtual functions, interfaces inherit interfaces, interfaces are uniformly registered, and calls are extremely discrete, thus achieving the efficiency of high-speed concurrent iteration. 2. Realize EI separation and skip IOC scanning. 3: P is responsible for reference and description, and C can carry out various functional operations through descriptions of multiple P. Mapping control technology ensures thread safety and stability. 4: V stores each single case class to ensure low data redundancy and unified recovery.

关于 VPCS 过多的描述, 我可以摘以前的一段笔记如下: VPC 架构编程思想, 经常长年的软件编程, 积累了一些对程序实现的思想, 通过达尔文进化论的认证, 一种有效地 VPC 编程理念基于 MVC+MVP 的中性耦合文中进行阐述. V 是一个观测者模式, 类似于存储对象, 观测模型. P 是处理机, 对注册接口的处理. C 是控制机, 对注册接口的描述和分类. S 静态控制机器, 为什么用静态控制机, 优势:1: 因为 PC 的分离, c 模式的函数皆通过抽象的虚函数继承, 接口继承接口, 接口统一注册, 调用极度离散化, 达到高速并发迭代的效率. 2: 实现 EI 分离, 跳过 IOC 的扫描. 3: P 负责引用和描

述, 一个 c 可以通过多个 p 的描述来实行各种函数运算. 映射的控制技术保证线程安全稳定. 4: V 存储各单例类, 保证数据冗杂度低, 统一回收.

2. 3 DETA Business backend logic application, 德塔商业后端逻辑应用

In the whole year of 2019, the VPCS back-end engine gradually formed some standardized functions and papers, which were applied to the front-end, back-end, cache, database and other subsystems of DETA. My evaluation of them is that they are lightweight and extensible. VPCS is gradually integrated into the works of Yangliaojing and Huaruiji. Of course, there are many shortcomings, the biggest one is that they do not repair themselves. Although I designed the sleeper and hallkeeper mechanisms, these mechanisms are only the corresponding business logic units that I complete through decision trees, not humanoid evolutionary thinking. At least, I don't think they are humanoid intelligence. To be precise, at present, they are only artificial intelligence, a kind of artificial intelligence logic corresponding to AOPM and VPCS, but not the humanoid evolutionary intelligence logic that I want. So, I started to explore humanoid computing again. About the application principal description of VPCS, the author designed a paper as below:

2019 年 一整年, VPCS 后端引擎 逐渐形成一些规范化函数和论文, 应用在德塔 前端, 后端, 缓存, 数据库, 等子系统中, 我对他们的评价是, 轻巧, 可扩展. VPCS 逐渐集成在 养疗经 和 华瑞集作品中. 当然不足也很多, 最大的不足是没有做到自我修复, 虽然我设计了 sleeper 和 hallkeeper 机制, 但是这些机制只是 我通过决策树 来 完成相应的业务逻辑单元, 不是类人的进化思维. 至少我觉得不是类人智慧, 确切的说目前只是人工智慧. 一种 AOPM VPCS 对应的人工智能逻辑. 而不是我想要的类人进化智慧逻辑. 于是我又开始了类人计算探索. 关于 VPCS 应用原理描述: 作者设计了一篇论文如下:

VPCS Backend Theory And Its Application

Mr. Yaoguang. Luo

Outline: due to the development of the software acquisition and definition in what we use the code theory always in messy and unforeseeable status. A new method of the coding style like VPCS that will show in this topic paper, feel free to resonate with my imagination of the portrait—VPCS (Vision, Process, Controller, Sets) theory, fun yet? Not only this paper will gazer a big point how we show the constructions of the VPCS, you guys also sure to get lots of idyllic landscapes of the coding sections. While you got lots of the illness codes at the so messy fungus projects, I guess at this paper out where you are finding anxiously. Let’s catch more opportunity about how does the VPCS working, executing and scheduling in our software project and make the software fast, fast and safe! lets go, So the key words as below:

大纲:由于软件的发展, 在获取和定义方面我们所使用的代码理论总是处于混乱和不可预见的状态. 一种新的编码风格的方法, 如VPCS, 将在这篇专题论文中展示, 请随意与我想象的肖像- VPCS(视觉, 操作过程, 控制处理器, 静态集)理论产生共鸣, 有趣吗?不仅这篇论文将关注一个大的点, 我们如何显示构建的vpc, 你也一定会得到许多田园风景般的编码部分. 当你在如此混乱的细琐项目中得到许多病垢代码时, 我猜在这篇论文中你会感到不安. 让我们抓住更多的机会, 如何工作的vpc, 执行和计划, 在我们的软件项目, 使软件快速, 快速和安全! Let′s go, 所以关键字如下:

Quantum Sets, Concurrent Consumer, Vision, Scheduler, Threads, Surf.

Introductions

Let see the verbal keys, the first time you …, okay, get any sense? Sure, this paper is not talking about the human careers, truly about SOFTWARER, as a human, if you got my points, yes, cool! Make any sense? Let's see the landscape as below figure 1-1.

让我们看看语言键, 第一次你…, 明白了吗?当然, 这篇文章并不是在讨论人类的酒店职业生涯, 而是真正地讨论作为一个类人的软件生命存在, 如果你明白我的观点, 是的, 很酷!任何意义?让我们看看如下图 1-1 所示的景观.

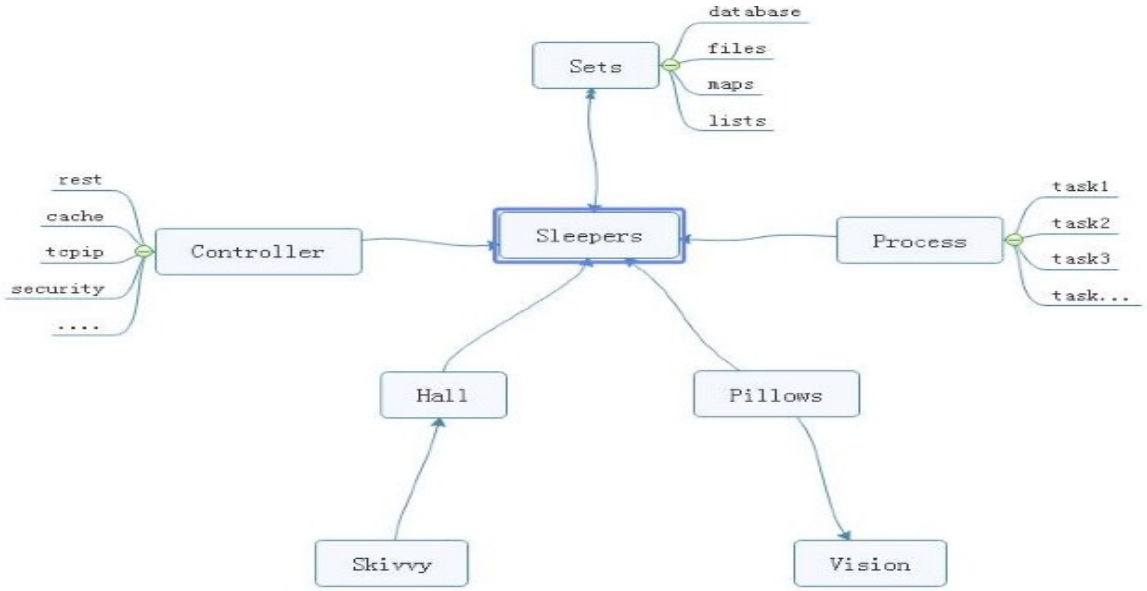


Figure 1-1 VPCS STAR MODE

From the ordinary software development architecture, always like a factory mode, for instance, controller, transaction delegate, web service, job bean, data DAO, like that of traditional backend or front-end coding style, but, compare now the seamless client's services system, those mode more and more not suitable for us for the project application, at least in the light level, multitasks, satellite boots projects system, if we choice the factory mode, you will feel so heavy. But the big conflict problem is where the factory mode was used in all and all bazaar companies. Even more CTOs that I met before often complaining about the reference room likes that "we need one server for database system, one more for cache system, one more for front end, one more, for backend, one more…." after that what do you think? My lord…

从普通的软件开发架构来看, 总是像一个工厂模型, 例如, 控制器, 事务委托, web 服务, 工作 bean 数据通道, 像传统的后端或前端编码风格, 但是, 对比现在, 无缝的客户服务系统, 这些模型越来越不适合我们项目的应用程序中, 至少在轻微水平级别, 多线程事务的同时, 卫星集群般部署的项目系统, 如果我们选择工厂模型, 你会觉得很沉重. 但最大的冲突问题是, 工厂模式被用于所有的集市公司. 更多的 cto 经常抱怨资料室, 比如"我们需要一个数据库服务器, 一个缓存服务器, 一个前端服务器, 一个后端服务器, 一个……", "那之后你觉得怎么样?"我的主…

Finding a new method of how to integrate the sets about the micro satellites service in the same sever, and make them small, lightly and faster for the commercial service, now become a fatal topic. Which can be a pretty warm-up for where I make an explanation for VPCS. The VPCS mode, only includes four aspects. Vision, Process, Controller, Sets, and those

factors makes an interaction in the sleeper containers. Let talk about the definition of the sleepers. From the software engineering domain, the sleepers are more like an identified thread person. Who can make a lot of fantasy-dreams in a Hall, what means a dream? Dream is a requirement what the consumer really needs to finished. But here the dream can be separated out more tasks, those tasks will register the ID in the Pillow, so that the sleeper hugs the pillow then goes into the hall and make a dream. Got an idea? Cool. So, what does the sleeper does in a hall? The answer is to make all kinds of the dream. For example, if we want to build the web service to get rest call, and return the JSON feedbacks, we only need to do like the way: Firth, build rest call path in the controller; Second: register the call requirements as a dream; Third, build the sets of the dream in the pillow, fourth hire a sleeper to hug this pillow, and go to the hall to make a dream process. At last, but least: return the dream goods. Any sense? Cool! For this unique instance, you will know that the sleeper was more like a socket, and the hall more like a thread pool, the pillows like the single visionary instance, and the sets like a visionary storage, the controller and the process those two sections is a common way of the factory mode. The steps landscape of the sleeper who makes a dream as bellow figure 1-2.

如何在同一服务器上集成微卫星蜂群服务设备, 使其小型化、轻量化、快速化, 成为当前重要的课题. 这对于我解释 vpc 来说是一个很好的热身. VPCS 模型, 只包括四个方面. 视觉、过程、控制器、设置以及这些因素在轨枕容器中进行交互. 让我们来谈谈梦想家的定义. 从软件工程领域来看, "梦想家休眠者"更像是一个已确定的线程 人员. 谁能在大厅里做很多幻想的梦, 梦是什么意思? 梦想是消费者真正需要完成的一项要求. 但是在这里, 梦可以被分离出更多的任务, 这些任务会在枕头上登记 ID, 所以睡觉的人拥抱枕头, 然后走进大厅做一个梦. 有一个主意吗? 酷. 那么卧铺在大厅里做什么呢? 答案是做各种各样的梦. 例如, 如果我们想要构建 web 服务来获取 rest 调用, 并返回 JSON 反馈, 我们只需要像这样做: 首先, 在控制器中构建 rest 调用路径; 第二: 把呼叫要求登记为一个梦; 第三, 在枕头上搭建梦境的布景, 第四, 雇一个睡觉的人来抱这个枕头, 然后去大厅做一个梦的过程. 最后, 但最重要的是:把梦想实现的结果输出. 任何意义? 太酷了! 对于这个独特的实例, 您将知道睡眠更像是一个套接字, 和大厅里更像是一个线程池, 枕头像单视觉实例, 像是和集存储、控制器和过程这两个部分工厂的模型是一种常见的方式. 做梦的人的台阶景观如图 1-2 所示.

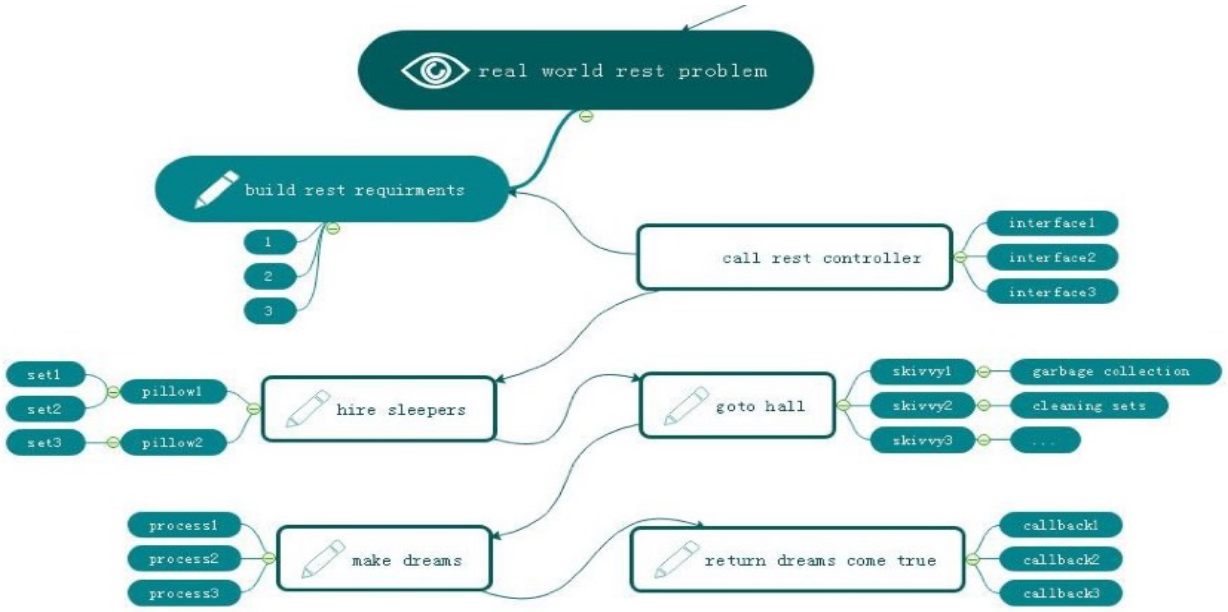


Figure 1-2 VPCS BACKEND MODE

Focus on this landscape, mostly different to the MVC: Mode View Controller, MVP: Mode View Presenter or other architectures we know before. but is very easy to understand after you read for a while. Too simple. Sleeper makes dreams come true, hall container sleepers, skivvy make up the hall, pillow clear and wake up the sleepers who often lost in finding the way in the dream. Got fun here, but I would hear more argue voice DETAils of my VPCS, desktop App once said:

VPCS is good in the concurrent WEB project, but not suitable for the desktop applications. Ok, follow this question, let make a new landscape based on desktop application as below figure 1-3.

专注于这个景观, 主要不同于 MVC: Mode View Controller, MVP: Mode View Presenter 或我们之前知道的其他架构. 但是读一段时间后就很容易理解了. 太简单了. 睡眠者使梦想成真, 大厅集装箱睡眠者, skivvy 使大厅, 枕头清楚, 唤醒经常在梦中迷路的睡眠者. 在这里得到了乐趣, 但我将听到更多的争论声音细节关于我的 vpc 桌面应用, 曾经有人说:vpc 是好的并行 WEB 项目, 但不适合桌面应用. 好的, 按照这个问题, 让我们创建一个新的基于桌面应用程序的横屏, 如下图 1-3 所示.

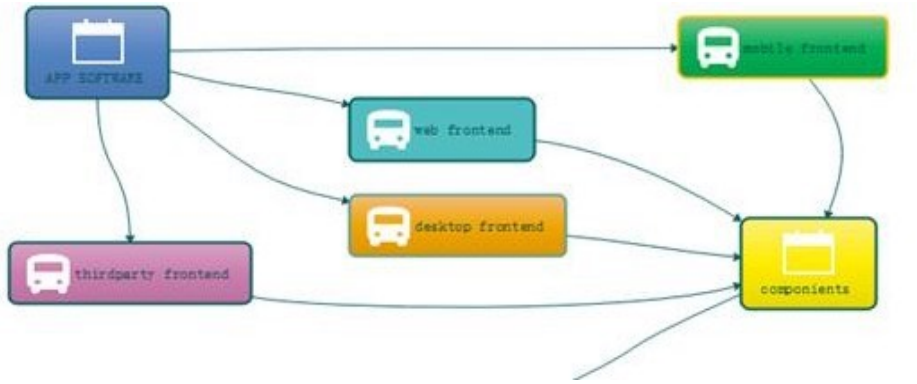


Figure 1-3 VPCS WORK WITH FRONTEND

From this picture, we know all of the software can be fast and safe while using VPCS, because It is already separated out the big system into backend and frontend two parts. And VPCS keeps safe and fast in the backend section. Compare to the MVC, VPCS will get more cautious and DETAils, and compare to MVP, VPCS also will get more safe and high efficiency. Those factors are why I will make inauguration here. In the common software engineering cycle life times, scientist used to build front end and backend for all kinds of the software applications, because it is easy to control. Why? Frontend only spend time to make design, and Backend for the data operations. Using VPCS system, we don't care about what they do for the front end, we only fit about what they want. Alignment that gets a blame and fix, then return OK, the restful service developer makes a voice that http functions are concurrent functions. At here, VPCS will say: concurrent functions are safe functions. We guess in the future REST-VPCS will be used in multiple WEB service. Especially in the high speed, efficiency, micro web systems with high level security for example medicine, DNA, cloud server, electronic police system and ecommerce systems etc.

从这个图中, 我们知道所有的软件在使用 vpc 时都是快速和安全的, 因为它已经把大系统分成后端和前端两部分. VPCS 在后端部分保持安全和快速. 与 MVC 相比, VPCS 会更加谨慎和细致, 与 MVP 相比, VPCS 也会更加安全高效. 这些因素就是我在这里就职的原因. 在常见的软件工程周期生命周期中, 科学家常常为各种软件应用程序构建前端和后端, 因为它易于控制. 为什么? 前端只花时间做设计, 后端只花时间做数据操作. 使用 vpc 系统, 我们不关心他们为前端做什么, 我们只适合他们想要的. 得到责备和修正, 然后返回 OK, restful 服务开发人员发出声音说 http 函数是并发函数. 在这里, vpc 会说: 并发函数是安全函数. 我们猜测将来 rest - vpc 将被用于多个 WEB 服务中. 特别是高速、高效、高安全级别的微 web 系统, 如医药、DNA、云服务器、电子警察系统、电子商务系统等. 最近养疗经引擎也开始基于 VPCS 进行布局. 其子系统已全部采用 VPCS 结构.

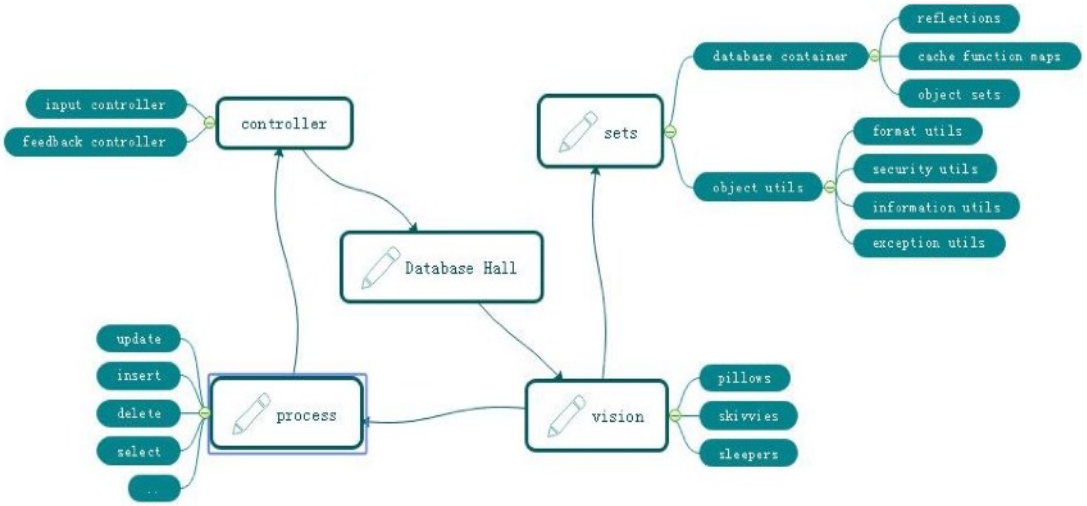


Figure 1-4 VPCS FOR DATABASE SYSTEM

As the figure 1-4, a new method of the Database system designing shows us VPCS is a pretty way for the modern data information management system. Definitely used in the DETA Database system. For this instance, do we get a view that the controller section of the factory mode becomes thin yet? Controller only works for the hands transactions, for example that the controller gets an input requirement such as select SQL, then immediately call the hall keeper to register this SQL and hire new sleepers to make a result. Because of the VPCS. Once it happened any exceptions, will very easy to awake sleeper and let them get theirs working papers out, finally call skivvy to fork the sets to the fresh sleeper. This method mostly be like a Count Down Latch mode, once the sleeper gets the dreams come true, then told the hall keeper for the feedback, hall keeper will make a type procession to return after everything goes well, this method mostly be like a Cyclic Barrier mode.

如图 1-4 所示, 一种新的数据库系统设计方法向我们展示了 VPCS 是现代数据信息管理系统的一种很好的方式. 明确用于 DETA 数据库系统. 对于这个实例, 我们是否得到了工厂模型的控制器部分变薄的视图? 控制器仅适用于hands 事务, 例如, 控制器获得一个输入需求(如 select SQL), 然后立即调用大厅管理员注册这个 SQL 并雇佣新的休眠人员来生成结果. 因为 vpc. 一旦发生任何异常, 会很容易叫醒睡眠者, 让他们拿出自己的工作底稿, 最后叫 skivvy 把餐具叉给新的睡眠者. 这种方法大多类似于一个倒计时锁存模型, 一旦睡眠者的梦境实现, 然后告知大厅管理员进行反馈, 一切顺利后, 大厅管理员会做出一个类型的队列返回, 这种方法大多类似于循环障碍模型.

Questions

How does skivvy doing? please see the figure 1-5 the hall building needs a single instance like a home keeper but here is a hall keeper, any else, this person is very important for keeping the VPCS safe, because all of the skivvies will be managed by him. You will see, the memory check, JVM garbage collection, disk cleaning, thread status management, deadlock alarm, security protocol all and all in one at here. Mostly like a static class in the VPCS system. If we need to know every thing about skivvy's work status, ok just call the hall keeper.

skivvy 做了些什么事务? 请见图 1-5, 大厅建筑需要一个像管家一样的单例, 但是这里有一个管家, 其他的, 这个人对于保持 VPCS 的安全非常重要, 因为所有的 skivvies 都将由他管理. 您将在这里看到, 内存检查、JVM 垃圾收集、磁盘

清理、线程状态管理、死锁警报、安全协议, 所有这些都集于一身. 大部分类似于 VPCS 系统中的静态类. 如果我们需要知道 skivvy 的工作情况, 可以打电话给大堂管理员.

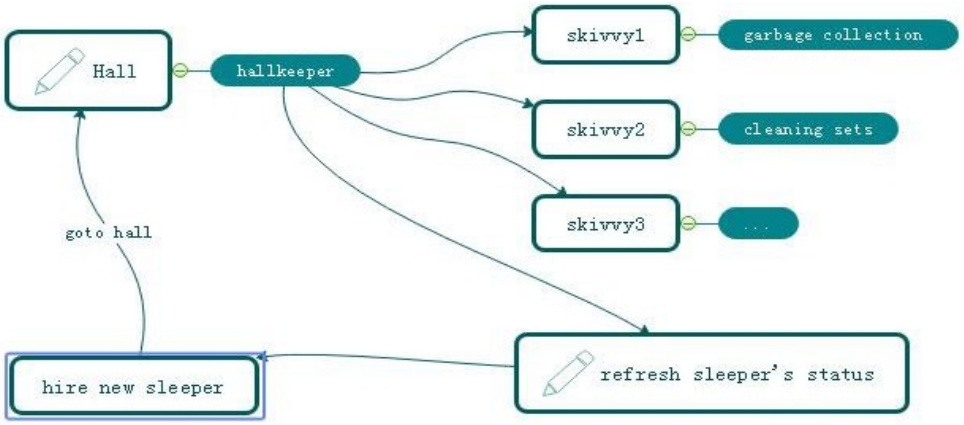


Figure 1-5 VPCS KERNEL

How does sleeper do? Make a dream? Cool, you shoot! in the VPCS system, it doesn't have the definition of the process, everything likes subsets. Immutable or unlined, hall keeper get request from visionary and hire the sleeper, who is likes a thread, get requirement, add those sets in pillow, hugs pillow then go to hall to make a dream, after that then return the callback to hall keeper what they did. Fun yet?

睡觉的人怎么样? 做一个梦吗? 酷, 你正中目的! 在 VPCS 系统中, 它没有进程的定义, 一切都像是子集. 不可变的或无线条的, 厅管得到梦想者的请求, 雇佣睡眠者, 谁是线程, 得到要求, 添加那些套在枕头, 拥抱枕头然后 去厅做一个梦, 然后返回回厅管他们做了什么. 有趣吗?

What does the sets mean? Sets, is a format of the data was appearing in the VPCS system. For the static prototype, it used like a concurrent hash table, and list which can be copy base on writing format, the single instance, it always runs in the static function or be liking an interface implementation because need safe at the same time, so that compare to the factory mode, it is too simple and without annotation. Everything becomes easy in this environment.

这些集合是什么意思? 是出现在 VPCS 系统中的数据的一种格式. 静态原型, 它像一个并发的哈希表, 使用这些列表, 可以复制基本写作格式, 单一实例, 它总是运行在静态函数或像是一个接口实现, 因为需要安全的同时, 与工厂模型相比, 它太简单, 没有注释. 在这种环境下, 一切都变得容易了.

The one more question is that so many peoples asked me what does the sequence diagram of the VPCS, because they really want to know why VPCS is faster and safer. Ok, please see the figure 1-6, the answer is absolutely, VPCS main components of the time sequence only contains five aspects. Almost similar like the hotel management. Certainly, we are talking about VPCS software, not for guesthouse. You will see that the rest call only makes the interactions with the hall keeper. And hall keeper got two jobs, one for waiting the fresh sleeper and one more for giving task to skivvy. The sleeper only hugs the relate pillow and make the dreams come true. Fun yet? Cool. VPCS only take cares about how does the sleeper's

imagination and skivvy's working status. If is the pillow broken? Make new pillow, got lazy sleeper? Get out his working papers, got a cheat skivvy? Fix of fire him, the real source of the java version project for the VPCS only 30kb, we will find more sources or documents from the reference links at the end.

还有一个问题是, 很多人问我 vpc 的序列图是什么, 因为他们真的想知道为什么 vpc 更快、更安全. 好的, 请看图 1-6, 答案是绝对的, VPCS 主要组件的时间序列只包含五个方面. 和酒店管理差不多. 当然, 我们说的是 vpc 软件, 而不是宾馆用的. 您将看到 rest 调用只与大厅管理员进行交互. 厅长有两份工作, 一份是等新来的人, 另一份是给佣人分派任务. 睡觉的人只会抱着枕边的枕头, 让美梦成真. 有趣吗? 酷. VPCS 只关心睡眠者的想象力和skivvy 的工作状态. 如果枕头被打破了? 做新枕头, 遇到睡懒觉的梦想家吗? 把他的工作文件拿出来, 有个服务骗子吗? 修复或者炒了他, 真正的 java 版本项目的源代码为 VPCS 只有 30kb, 我们将在后面的参考链接中找到更多的源代码或文档.

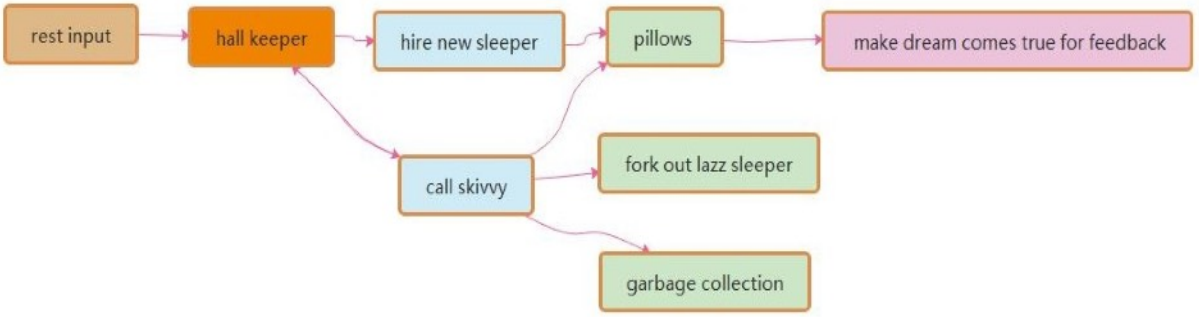


Figure 1-6 VPCS Sequence Diagram

I always be asked by the colleagues that once said: how does the hall build? I answered them, such like the hospital, no one cares about the address of hospital, because they just call the cell phone number when will get a directly feedback. This is why I need a hall keeper role in the gate way. For the instance about figure 1. 6. 1, this sample is a true demo in the real world for the WEB rest service. Its very important to create a player role such like hall keeper. What would like about author's theory? Because of the maintenance. Because of whom, the software build team are very easy to make a maintenance web portal, all of the system current status will be solved on this html page by DEVOPS.

我总是被问道 VPCS 中的 大堂经理用来干什么, 我回答说 正如一个医院, 没人关心医院的地址, 因为他们拨打 120 救护车就来了. 大堂经理就像这个接电话的. 后端接口请求同样, 没有哪个事务逻辑需要知道怎么被执行, 因为大堂经理按照索引规范已经登记好了. 这种登记方式非常容易被开发运维人员维护操作.

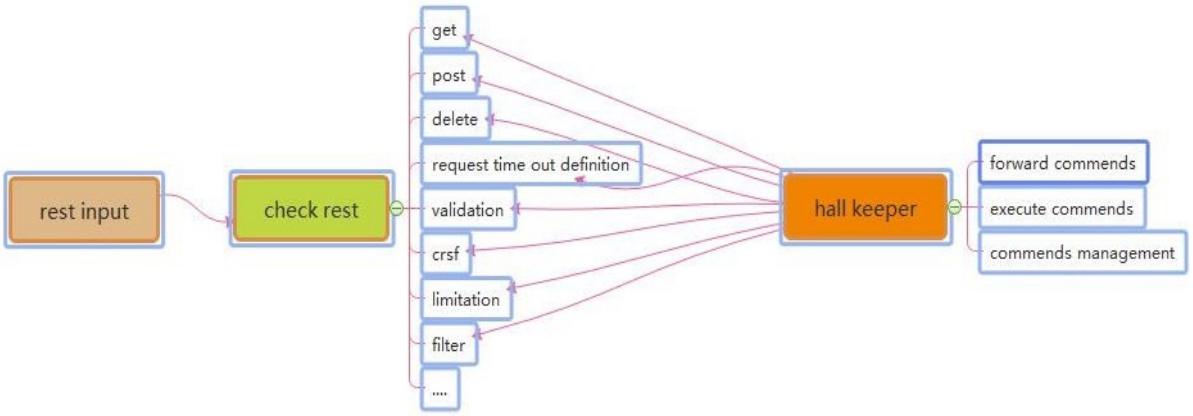


Figure 1-6-1 The Interaction Between Rest Call and Hall Keeper

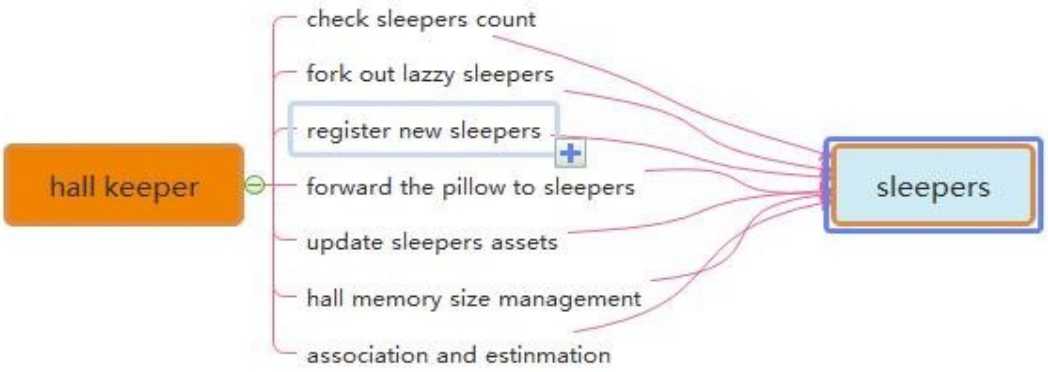


Figure 1-6-2 The Interaction Between Hall Keeper and Sleepers

Many of these software developers also asked me how and why we fork out the LAZZY sleepers excluding their sets. Author answered because of the pillows. When the sleepers be hired from the hall keeper, they will get an independently pillows such like static functions. So that sleeper only has their own identify attributes and unique information as the singe instance class. Once they got theirs working paper, the pillows they used will be arranged to the new fresh sleeper, this theory keeps safe, quality and quantity. Like figure 1. 6. 2. 1 VPCS kernel.

很多软件开发者同样会问我 如果删除掉一些睡懒觉的线程, 他们的枕头怎么处理? 这就是 VPCS 的闪光点, 因为枕头是 id 标识的静态数据, 所以, 是可以重用的. 新的梦想家可以从这些枕头里提取有用的数据碎片. (当然这里是软件不是酒店, 酒店的枕头是要高温清晰消毒的^-^)

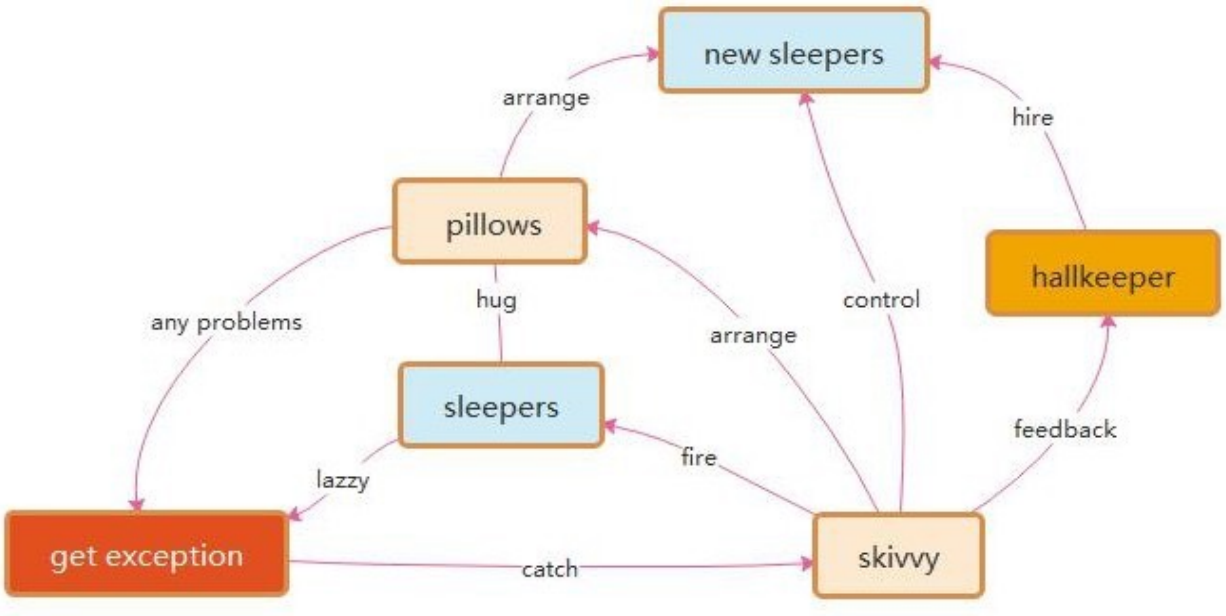


Figure 1-6-2-1 VPCS kernel

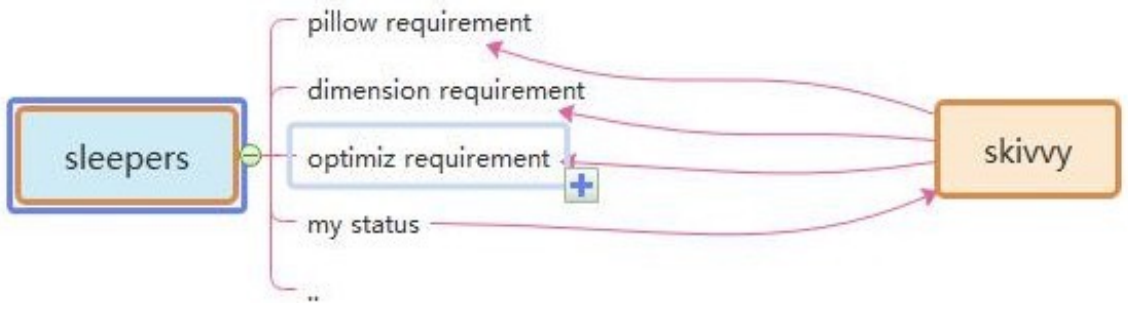


Figure 1-6-3 The Interaction Between Sleepers and skivvy

Many times, I got questions about the DEVOPS, they really worry about VPCS if suitable for their project system maintenance? The answer is absolutely. How does the VPCS supports us for daily works? Because of Hall Keeper and skivvy, as figure 1-6-3 and 1-6-4. DEVOPS will get all of this transparency information about project from hall keeper under the encryption and security issues. Also, hall keeper will directly get the rule for DEVOPS by rest calls, then makes commend to skivvy. All of the information and record logger will be cached by hall keeper, that keeps controllability. The html control page will make an interaction between hall keeper and DEVEOPS, which keeps safe, implementation capacity, transparency and maneuverability. This is my true answer.

很多时候, 我被一些运维问道, 采用了VPCS 架构怎么进行系统维护? 答案是很明确的, ray 先生曾问我 作为运维他们往往纠结于 耦合度, 透明度和可维护性 VPCS 具体是怎么体现的? 我的回答是 大堂经理与服务员的命令落实, 这些落实情况可以进行完整的可观测的日志记录, 内存状态, 中间属性标识.,

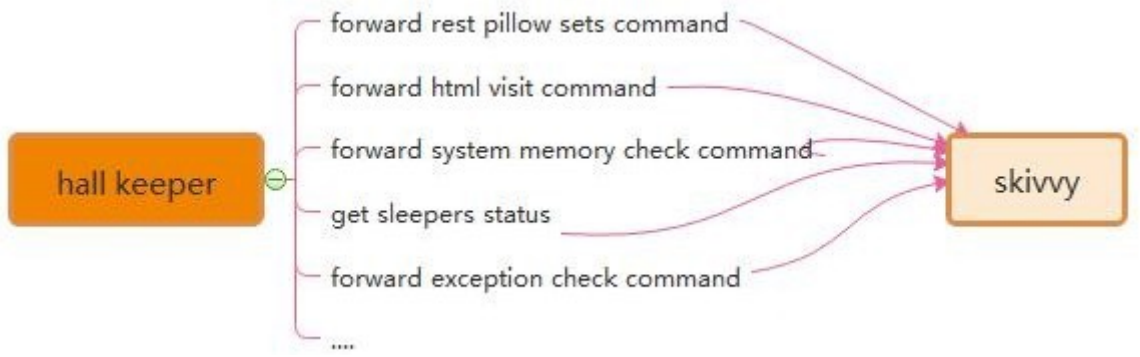


Figure 1-6-4 The Interaction Between Skivvy and Hall Keeper

Recently about the separation-issue between controller and process. Following this topic, my answer that the key is the separation between pillows and sleepers. Due to the pillows all have their own unique ID, skivvy will easily arrange the pillow to new sleeper after the original sleeper who made problems. Make unique ID and arrangement by ID, is the key method. Also, for the rest call service, the asymmetrically irreversible combination encryption is one of the best solutions to the data leakage controller. VPCS seems so smart.

最近 关于 VPCS 能用在 IOS 中么? 因为它担心 pc 分离 有可能 导致 数据的溢出, 我的回答是绝对的安全, 因为每一个线程都进行了唯一的公有标识, 连静态属性, 单例属性都有明确的属性表示, 这个标识能在内存中时时刻刻着找到它的状态. 另外 rest 的异步消息握手同样能够进行精准的维护.

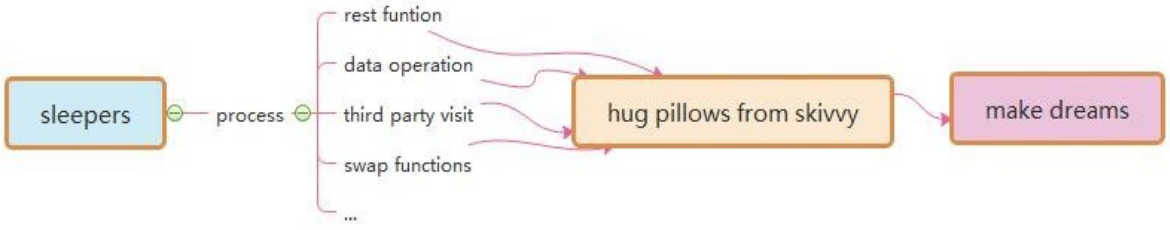


Figure 1-6-5 The Interaction Between Sleepers and Pillows

3 DETA Catalytic computing

I got my mind storm for a month in early 2019. How to realize human computing? It's been bothering me for a long time. How to start? I didn't have a clue, so I started to read my notes made in the past 20 years. I got it! Do basic research! According to my notes, I dig some unknown basic knowledge. I am very happy, because I have the results of quick sorting by left-right comparison in 2014, the butterfly calculation manuscript of Fast Fourier, the Chinese word segmentation works of Huaruiji, Unicorn ETL, socket stream PLSQL database, etc. And an idea came into being. I think about optimizing them continuously, refining, optimizing and testing repeatedly, and remembering these optimized ideas.

2019 年我迷茫了一个月, 类人计算, 怎么实现? 困扰了我很久, 怎么开始? 我没有头绪, 我于是开始翻阅我这20 年做的笔记, 有了! 做做基础研究! 根据我的笔记来挖掘一些未知的基础知识. 我非常的快乐, 因为我有 2014 年的左右比对法快速排序的成果, 快速傅里叶的蝶形计算原稿, 华瑞集中文分词的作品, Unicorn ETL, socket 流PLSQL 数据库 等, 一个想法应运而生, 对 持续专注的优化它们, 不断地重复的细化, 优化, 测试, 记住这些优化的思想, 这种思想, 我思考了很久, 一定是类人的进化思想.

3. 1 DETA Catalytic computing history, 德塔催化计算历史

Since of the bright flashes, may I follow the operation method. The refinement method of DETA's first catalytic calculation is first reflected in DETA parser, such as the refinement of semantic part-of-speech analysis, the optimization of flow valve, the irrational conditional transformation of discrete data, the filtering of the same frequency operator, and the filtering of calculation peaks. These optimization methods of human thinking gradually form a system, which not only changes the design mode of DETA's works, but also changes the author's research and development philosophy.

既然想到了闪光点, 那么就应该需按照操作方法. 德塔的第一篇催化计算的细化方式首先在德塔分词中, 逐渐体现, 比如 语义词性分析的细化, 流水阀门的优化, 离散数据的非有理条件变换, 同频算子的过滤, 计算高峰的过滤, 这些类人思考的优化方法, 逐渐形成一种体系, 不仅改变德塔作品的设计模式, 同时也在改变作者的研发理念.

3. 2 DETA Catalytic computing development, 德塔催化计算发展

R&D is not successful every time. In the process of butterfly calculation optimization of Fast Fourier, I coded the features of discrete DCT in complex numbers, which took me one month, but failed. I remembered that I said in Weibo at the beginning that I could speed up the calculation of Fast Fourier by 200 times, but I really didn't give up. Since butterfly calculation optimization was unsuccessful, I tried to sort the small peaks by fast left-right comparison. I was excited when I saw the 10th generation of single machine random double with a sorting speed of 12 million arrays per second of quick sorting. My thought is right, and thinning logic is an important way of human thinking. Here, the author designed an argumentation paper when designing fast word segmentation and extremely fast peak filtering catalytic sorting, as follows:

研发并不是每一次都是成功的, 在快速傅里叶的蝶形运算优化过程中, 我将离散 DCT 的特征进行复数编码, 花了我 1 个月, 结果失败了, 想起刚开始时候我在微博里说我能将快速傅里叶计算速度加快 200 倍, 着实打了脸, 我并没有放弃, 既然蝶形计算优化不成功, 那么在快速左右比对小高峰过滤排序上试试? 当我看到单机随机 double 每秒1200 万数组排序速度的第 10 代及快速排序出来的时候, 我振奋了. 我的思想是对的, 细化逻辑是类人思考的一种重要方式. 在这里作者设计快速分词和 极快速小高峰过滤催化排序时候设计了一篇论证论文如下:

Theory on YAOGUANG's Array Split Peak Defect

Mr. Yaoguang. Luo

Outline: In the common software development factory, engineer always did more and more interactions with data structure and math algorithms. Especially in the recursion, convolution, sort and generic loops, scientists liked to find a simple, more sufficiently and alignment way to face the project requirements with the large association. For instance, me, I really got a real-world problem at this domain while I use quicksort, also for another project such like DETA parser. What is the peak array split defect? How does It count the real-world problems? Why need find It and how to get the nice solution? Cool, this paper will cause an implementation about our goals, ok now, keep forward to the context where I talking as below, thanks for more theory, DETAils and the source code implementations please check the bottom reference section.

在一些主流的软件研发企业, 工程师总是花费很多时间在数据结构和数学算法上, 特别是在递归, 演化, 排序, 和常规循环. 科学家因此想发现一种简单高效的算法来解决一些需求, 比如我, 如快速排序, 德塔分词, 什么是非对称数 列切分, 这些逻辑怎么解决当前真实的社会问题? 这篇文章能够解释许多问题, 丰富的论据和论证以及论证的源码我都会毫不保留的提供.

Peak, Array, Split, Defect, Recursion, Convolution, Sort, Generic

Goal one: Quicksort Yaoguang. Luo 4D

DETAils:

For example the array input as below where we gave 11 digits.

关于基数拆分初始, 给定 11 个数列 队,



1. The first split, we could see the digit-6 will auto arranged to the right part.

2.

第一次拆分如上, 我们会发现 6 被默认在右方.



3. And the second split, we may see the digit-3 will be auto arranged to the right part

第二次拆分, 发现 3 在右方.



4. The third split, we may see the digit-4. 7. 10 will be auto arranged to the right part

第三次拆分, 发现 4, 7, 10 出现在右方.



2.Thinking:

After the split array showing, we could see clear that the big problem about the asymmetry defect, as I did an annotation of N, so the i of N will absolutely find a n/POW (2, i) value points, as an insufficiency asymmetry defect mode, I fall in thinking…if I do any compute theory as the same with this mode style, for example in the recursion or inner loops, it will autonomic separate to the 2 different process way, it necessary to do indifferent flows.

通过上面的迭代微分, 可以发现一个有规律的非对称非基数缺陷问题. 我用数学标识数组 n, n 为基数, 则 n的第 i 次拆分, 关于这个 n/pow(2, i)的缺陷数, 呈现不饱和对称模型, 这个细节我引起了思考如果, 我在做函数递归, 2 分迭代, 和矩阵微分的时候, 出现了类似的基数划分, 我有必要考虑通过 2 种算法内核形式来处理基数列和偶数的计算.

3.Problems:

So, after the above thoughts, I may get any flashes, First, the even and odd digits both are asymmetry while in the Differential loops. For this noise, I defined as (Tinoise Peak) Second, once we did a split compute under this mode, It must get more unfair sets. I defined as (Tinsets defect). Third, if this mode almost in the messy and timer data system, It will catch more time and asserts wastes or exceptions.

通过这个实验, 我得到一些答案: 1 不论是基数还是偶数都会增加不对称, 而不是仅仅是基数的问题. 这个噪声, 我定义为罗瑶光拆分噪声峰. 2 拆分的基偶划分, 必然会造成内部计算算子的不平衡, 我定义为罗瑶光算子缺陷. 3 如果这个缺陷在海量数据处理中有时序性, 那么这个算子缺陷会产生巨大的计算浪费.

4.Solutions:

For the god like, I find three solutions while I currently enrolled in my projects. First: computer logic acceleration, at least It can avoid the waste of the compute by using inner process optimism. -- To avoid the deep recursion. Second, reduce the compute sets. For any less memory system, we may reduce more and more memory garbages after we reduce the inner

register or temp value sets. Third, we may make an optimization of the function logic where to instead the old complex functions. Those ways include the condition, algorithm, method or discrete optimization. End, we may use mathematics of double differential, deep definition, acquisition or polynomial to get the solutions.

这个细节体现在罗瑶光小高峰过滤快速算法中. 1 增强计算逻辑, 我们可以通过增加函数内部的计算性能来弥补深度迭代浪费. 2 减少计算算子, 我们通过减少计算临时变量和算子来避免内存垃圾增量. 3 化简计算条件, 我们可以通过函数优化, 条件优化, 来化简计算逻辑. 4 重微分迭代, 我们也可以进行微分, 重微分, 条件微分, 迭代来进行多项式展开计算.

5.True Instances, let me show the algorithms here,

```
private int partition(int[] a, int lp, int rp) {//第5版本 topsort 为int x= a[lp]<= a[rp]? a[lp]: a[rp];

        int x= a[lp]< a[rp]? a[lp]: a[rp]; //reduce the compute values, reduce the
        recursion peak int lp1= lp;

        while(lp1++ < rp){

                while(!(a[lp1++]> x|| lp1> rp)) {// reduce the condition differential check, reduce the recursion
                loops

                }

                while(a[rp-- ]> x){}

                if(-- lp1< ++rp){

                        int temp= a[rp]; a[rp]= a[lp1]; a[lp1]= temp;

                }

        }

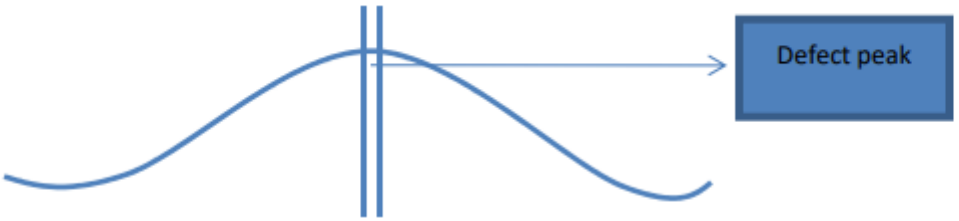
        a[lp]= a[rp]; a[rp]=
        x; return rp;

}
```

From this code: in a common quicksort way, the recursion based on the average deep split, suppose the initial array length is N {1, 2, 3…n} is an Odd, so the separate two arrays will cause an asymmetry defect, those timer asymmetry compute peak collection will cause more and more probability problems such like jam, lock, time waste and heap increment.

这段代码 说明在传统的快速排序中, 递归总是基于平均的深度分裂, 如果产生非对称的 迭代函数, 我们需要讨论一些算法来过滤这些计算概率高峰, 如 呼叫拥堵, 时间损耗, 内存堆增量等问题因素.

The odd peak binary split as below:



How to avoid those timer distinction peaks? I go more absolutely research where focus on these problems, first, differential flows. This flash is not suitable for here, May good for the DETA parser, I will show you later. Second, compute acceleration. Yep, this is a good way, for example find the big X as the code blew, It will cause the while loop ability accelerations.

怎么避免时间高峰? 我有一些明确的逻辑, 首先流微分, 其次是计算加速, 和循环优化加速.

```
int x= a[lp]< a[rp]? a[lp]: a[rp];
(int x= a[lp]< =a[rp]? a[lp]: a[rp];)适用于string swap top sort 5D
```

Third, De Morgan condition differential as the code below, It will cause the condition ability accelerations.

第三, 离散定律变换 也能进行决策条件优化.

```
while(!(a[lp1]>x|| lp1>=rp)) {
```

At last, but the least, value reduce, code optimization both are very important way of the peak avoid filter.

最后至少, 算子减少, 代码优化都能有效地进行计算小高峰过滤.

Goal Two: DETA parser

6.DETAils:

Last year I help my father to develop the study software about getting the medicine data collection for quick search. I' m going to try to build a search engine system, input format is a string, how to get a Chinese string array split?

去年, 我帮我父亲研发学习软件, 关于医学数据的迅捷搜索. 在设计搜索引擎的过程中, 我遇到了一个问题就是格式化字符串中有效地拆分中文词汇.



convolution length indicate by marching Nero index tree as below: 2|1|2|3

如图, 演化的开始按照神经森林词汇索引进行面切分, 如下.



convolution POS indicate as below: n |c |n |adv |adj

切分后进行面中的词性切分如下



convolution split

词性切分后进行组合如下



7.Thinks:

While I use this way on my DETA project Chinese separations, I met so many problems, the more and more important problem is the POS frequency peak waste, my POS flow functions will spend a lot of time to do the low prior convolutional split condition check first… It cost me a lot of time… after I did a collection of my projects, the results are clear.

思考了下, 当我用这种方法进行德塔工程, 我遇到了许多问题, 特别是词性拆分算法的频率峰, 很多时候高频使用的函数总在不常用的函数后面, 造成了时间浪费, 于是开始频率阀门排列优化.

8.Problems:

First, convolution kernel gets asymmetry problems. Second, unnecessary conditions check loops. Third, unimportant heap register values. End, values sets and conditions sets too more.

首先, 演化的内核 有许多非对称的问题, 其次, 不必要的条件和循环检查, 然后 不重要的堆注册的变量繁琐, 最后静态算子, 条件算子过多.

9.Solutions:

First, format the convolution kernel of the index dictionary tree, for example I use the char ASCII as the index length to reduce the match time of the convolution length indication. Also, I define the POS convolution kernel size less than five. Second, I did a condition frequency statistic, and re arrangement it, at the same time, reduced a lot of the inner sets to avoid the compute pause.

首先, 格式化索引树内核, 比如 ASCII 索引, 按长度划分, 同样, 词性内核我定义长度不超过 5 个字, 第二我统计了执行条件函数的频率, 然后流水阀门重新排序了, 高频的放在前面. 依次排列. 同时, 尽可能的减少内部条件算子, 避免计算高峰.

10.True instance:

11.Finally I developed a convolution String array split way for marching as below: orange color are presplit sets

11.1. Check 4 chars slang

最后, 我设计一套内核方式, 字符串词切分先 4 字成语谚语比较



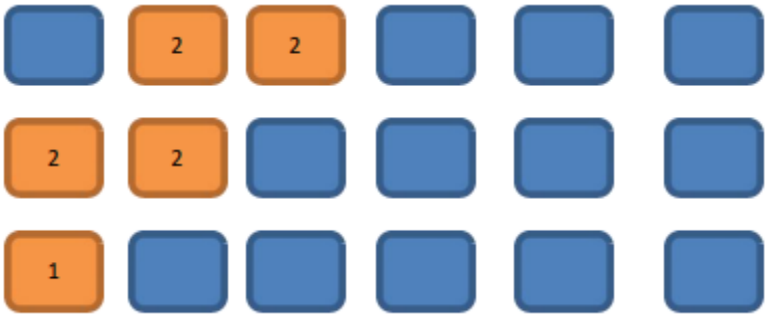
11.2.Check 3 chars key word

不是成语然后 3 字比较



Check 2chars normal word

然后 2 字比较



最后单词拆分

In order to make a compute acceleration, I did 2 string builder array to store the pre-order sets.

为了计算加速, 我设计了 2 个数组用来存储分词坐标的前缀 词储.

in order to make a PCA POS acceleration, I did 5 chars marching array to store the post order sets.

为了进行主要词性拆分加速, 我另外还设计了 5 个长度 后缀 字储.

It seems the Nero Index, NLP and POS for the PCA separation with convolution kernel marching by using stepwise iterative differentiation got much higher sufficiency.

计算拆分的高效性可以使用我的德塔分词的作品测试. 充分的数据论证其性能是优秀的.

3. 3 DETA Catalytic computing application, 德塔催化计算应用

In order to demonstrate the importance of DETAiled logic, I began to integrate this logic concept into my YangLiaoJing and all my soft works. When I saw 13 million high-accuracy word segmentation per second, 6 million mixed phonetic symbols per second, 12 million double arrays sorting per second, and other amazing works came out, I began to sigh my own cognition. I unreservedly opened up all these ideas and works, which hope aroused humanoid thinking resonance.

为了论证细化逻辑的重要性, 我开始将这种逻辑观念, 融入我的养疗经和我的一切软著作品中, 当我看到每秒1300 万的高准确度分词, 每秒 600 万的混合象契拼音笔画排序, 每秒 1200 万的double 数组排序, 等惊人性能的作品问世, 我对自己的认知开始感叹了. 我毫不保留的将这些思想和作品全部开源了. 引起人类的思维共鸣. 7 个月前我在对 dataswap 作品的评论中描述.

At present, the significance of differential catalysis has included seven categories: frequency valve (von Neumann) differential, discrete logic (De Morgan) differential, high-frequency function degradation, conditional refinement differential, executive mode differential, giant system (Qian Xuesen) module differential, and mathematical differential (Newton, Blainez), which is the only way to catalyze humanoid DNA evolutionary algorithm. And made a DETAiled demonstration and summarized as follows

微分催化的意义 目前已经包含了 频率阀门(冯诺依曼, 这里属于排队论, 严谨点不属于冯诺依曼)微分, 离散逻辑(狄摩根)微分, 高频函数降解, 条件细化微分, 执行模式微分, 巨系统(钱学森)模块微分, 数学微分(牛顿, 布莱尼兹)7 大类, 是类人 DNA 进化算法 催化 的唯一途径. 并做了详细的论证归纳如下

《微分催化计算作为类人 DNA 进化的唯一途径》 论证实例如下:

DETA 快速分词 POS 流水阀门微分算法 实例论证:

https://gitee. com/DETAChina/DETAParser/blob/master/wordSegment/org/tinos/engine/pos/imp/POSControllerImp. java

论证主题: 微分催化计算能非常好的观测到量子态函数的执行流水逻辑, 通过统计可以将高频函数逐步按冯诺依曼态提前, 低频的逻辑逐步淘汰筛选.

论证结果: 论证成功. 减少无关代码遍历次数. 大幅提高算能.

论证影响: 类人 DNA 进化算法的进化机制主要途径.

Demonstration of differential algorithm of POS water valve with DETA fast segmentation;

Demonstration topic: Differential catalytic calculation can well observe the execution flow logic of quantum state function. Through statistics, high-frequency function can be advanced gradually according to von Neumann state, and low-frequency logic can be eliminated and screened gradually.

Demonstration result: The demonstration was successful. Reduce the traversal times of irrelevant code. Greatly improve the computing power.

Demonstrating influence: the main way of evolutionary mechanism of humanoid DNA evolutionary algorithm.

罗瑶光小高峰计算过滤排序算法第4代实例论证： https://gitee. com/DETAChina/DataSwap/blob/dceeb0b06f726d640553964058d85b736354ac89/src/org/DETA/tinos/array/L YG4DWithDoubleQuickSort4D. java

论证主题: 微分催化计算能非常好的结合离散数学体系从数字逻辑层面进行微分过滤高频函数. 保证平滑

论证结果: 论证成功. 大幅增快计算速度.

论证影响: 类人 DNA 进化算法能有效平滑计算高峰.

Demonstration of the 4th generation example demonstration of filtering sorting algorithm for Luo yaoguang s small peak calculation;

Demonstration topic: Differential catalytic calculation can be combined differentially with discrete mathematics system to filter high-frequency functions where from the digital logic level. Ensure smoothness

Demonstration result: The demonstration was successful. Greatly increase the calculation speed.

Demonstration influence: humanoid DNA evolutionary algorithm can effectively smooth the peak of computation.

罗瑶光欧拉森林商旅环微分TSP算法第2代实例论证:

https://gitee. com/DETAChina/Data_Prediction/blob/master/src/org/tinos/DETA/tsp/YaoguangLuoEulerRingTSP2D. java

论证主题: 微分催化计算能非常好的将传统的复杂逻辑进行认知层的思维模式优化改变, 一开始便从根本上改变认知过程.

论证结果: 论证成功. 微分催化计算在一些社会领域能改变传统认知方式.

论证影响: 类人 DNA 进化算法能有效地选择最快的算法模块和认知模块来做特定领域的计算, 提高算能.

Demonstration of the second generation of differential TSP algorithm for Luoyaoguang Euler forest business travel ring Demonstration topic: Differential catalytic computing can optimize the thinking mode of traditional complex logic at cognitive level, and fundamentally change the cognitive process from the beginning.

Demonstration result: The demonstration was successful. Differential catalytic computing can change the traditional cognitive style in some social fields.

Impact of demonstration: Humanoid DNA evolutionary algorithm can effectively select the fastest algorithm module and cognitive module to do calculation in specific fields, and improve computing power.

罗瑶光象契字符串条件微分排序算法第7代实例论证： https://gitee. com/DETAChina/DataSwap/blob/master/src/org/DETA/tinos/string/LYG4DWithChineseMixStringSort7D. java

论证主题: 微分催化计算能可观的将条件函数统一, 减少逻辑复杂度, 并能持续优化和持续专注.

论证结果: 论证成功. 微分催化算法能很好地将整体函数的局部模块进行 VPCS 逻辑拆分, 形成 DNA 肽链的 INITON 运算嘌呤元.

论证影响: 类人DNA进化算法的自主进化机制保障.

Demonstration of the 7th generation example of Luo Yaoguangs conditional differential sorting algorithm for light image strings; Argument topic: Differential catalytic computing can unify conditional functions considerably, reduce logic complexity, and continuously optimize and focus.

Demonstration result: The demonstration was successful. Differential catalysis algorithm can split the local modules of the whole function by VPCS logic, and form the purine element of INITON operation of DNA peptide chain.

Demonstration influence: the guarantee of autonomous evolution mechanism of humanoid DNA evolutionary algorithm.

DETA Socket 流可编程数据库引擎的PLSQL语言微分编译机实例论证： https://gitee. com/DETAChina/DETA_PLSQL_DB/blob/master/java/org/lyg/db/plsql/imp/ExecPLSQLImp. java

论证主题: 微分催化计算 多条件 执行的 可用性.

论证结果: 论证成功. 微分催化算法能提供 函数系统运算时 可逆向可观测运维保障.

论证影响: 类人 DNA 进化算法的 条件微分, 逻辑微分, 高频阀门前置等功能的 综合集成应用实践.

Demonstration of PLSQL differential compiler for DETA Socket stream programmable database engine; Demonstration topic: Availability of multi-condition execution of differential catalytic calculation.

Demonstration result: The demonstration was successful. Differential catalysis algorithm can provide reverse observable operation and maintenance guarantee for functional system operation.

Demonstration influence: the comprehensive application practice of conditional differentiation, logical differentiation, high-frequency valve preposition and other functions of humanoid DNA evolutionary algorithm.

These arguments were a year ago. At present, many works have been in a good follow-up state because they are developed as subsystems of the project of YangLiaoJing. Over the years, I have been thinking, what is the final expression of DETA acquisitive logic? I have never stopped exploring, and I have always been absolutely focused.

上述这些论证是一年前的, 目前很多作品因为作为养疗经项目的子系统研发, 一直保持良好的跟进状态中. 这些岁月, 我一直在思考, 细化逻辑的最终表达方式是什么? 我一直没有停止探索, 持续地绝对专注.

4 DETA Finding initions

I have been thinking, what is the final expression of DETA acquisitive logic? I have never stopped exploring, and I have always been absolutely focused. I must find the final expression of these logic. From 2018 to 2019, I thought that the final expression of logic refinement must not be as simple as AOPM and VPCS. VPCS is just a refinement layer of AOPM, so how can VPCS be refined? So, I began to sort out my existing things, my works and soft thoughts. God, I can only make persistent and absolute focus on what I have. I have to make a bet.

我一直在思考, 细化逻辑的最终表达方式是什么? 我一直没有停止探索, 持续地绝对专注. 我一定要找到这些逻辑最终表达方式, 我 2018-2019 年认为逻辑细化的最终表达方式一定不是 AOPM 和 VPCS 这么简单, VPCS 只是AOPM 的一种细化层, 那VPCS 怎么细化呢? 于是我开始整理我已有的东西, 我的那些作品和软著思想. 上帝啊, 我只能在我已有的基础上进行坚韧地 持续地 绝对专注. 我得赌一次.

4.1 DETA Finding initions history, 德塔催化计算算子单元寻找历史

What's under VPCS? What is the essence of a function? At school, I got some basic answers. The primitives of DNA are ACGTU purine and pyrimidine, the primitives of back-end architecture are VPCS, the primitives of thing logic are AOPM, and the primitives of database are IDUC addition, deletion and modification. The primitives of function are IOAON Input, Output, And, Or, Negation, which I can only find in the knowledge structure I can understand and have. how to demonstrate? How to confirm the argument?